



MASTER THESIS

In Order to Obtain the

RESEARCH MASTER

in

Data Science

Presented and defended by:

Ali Alzein

On,, 2026

Title

LLM-Enhanced Graph-Based Intrusion Detection

Supervisor

Dr. Hussein Hazimeh

Reviewers

Dr. Hassan Harb

Dr. Ali Jaber

Abstract

Intrusion detection can be posed as a graph problem, where flows between hosts carry structure worth learning, or as a language problem, where a flow can be described in words a pretrained encoder understands. This report builds one edge-level pipeline that does both. Flows are aggregated into a directed multigraph over IP addresses. Each edge is encoded twice: once structurally by a graph attention network, and once as a label-free sentence read by a frozen cybersecurity language encoder. Four stages are compared on two NetFlow datasets whose class imbalance differs by roughly a factor of 47: each branch alone, gated attention fusion, and a feedback loop that consults the semantic branch about high-entropy edges and returns its judgement into the edge representations. Everything is measured at three training seeds. A difference counts as separated only when a bootstrap over edges and seed excludes zero.

Two results carry the work. Given a classification head trained on the semantic embeddings as its consultant, the loop is separated above every other rung and above every faithful re-trained baseline, on both datasets. Given the weaker embedding-only consultant it was first built with, nothing is separated at all. Second, with output fusion disabled, an oracle consultant converts the feedback channel into a separated gain on both datasets and no consultant we could build does. The binding constraint is the consultant, not the mechanism around it.

Keywords: intrusion detection, graph neural networks, edge classification, structural-semantic fusion, uncertainty-guided feedback, negative results.

Contents

1	Introduction and Context	4
1.1	Why combine graph and language representations?	5
1.2	Research problem and questions	6
1.3	Objectives and contributions	7
1.4	Report structure	9
2	Related Work	10
2.1	Graph-based network intrusion detection	10
2.2	Language models over security text and network traffic	11
2.3	Fusing structural and semantic representations	11
2.4	GNN and LLM combinations in intrusion detection	13
2.5	Iterative GNN–LLM feedback	13
2.6	Positioning	15
3	Method	17
3.1	Datasets and Common Processing Pipeline	17
3.2	Directed Multigraph Construction	18
3.3	Edge Feature Encoding	19
3.4	Structural Branch	20
3.5	Semantic Branch	21
3.6	Adaptive Gated Attention Fusion	22
3.7	Uncertainty-Guided Semantic Feedback	23
3.8	Baseline Protocol	25
3.9	Evaluation and Training Protocol	27
4	Results	29
4.1	The protocol as it was run	29
4.2	The model ladder	30
4.3	Feedback ablations and the mechanism in isolation	33
4.4	Five attempts on the embedding-only consultant	35
4.5	Attention injection	36
4.6	Re-trained published baselines	37
4.7	Per-class behaviour	40

4.8	Knob selection	41
5	Discussion	42
5.1	Answers to the research questions	42
5.2	Imbalance is the axis the two datasets differ on	43
5.3	Where the loop’s gain comes from	44
5.4	What the configuration buys, and what it does not	45
5.5	Limitations, read as findings	45
5.6	What follows from this	46
6	Conclusion	48
	Acknowledgements	50
	Statement on the use of AI assistance	50
	References	53
A	Negative mechanism results	54
A.1	Conditions A, B and C	54
A.2	What the canonical loop was actually injecting	55
A.3	Per-class reliability of the embedding-only consultant	55
A.4	The five treatments	56
B	Supporting tables	56
B.1	Per-class F1	56
B.2	Knob selection curves	56
B.3	Consultant complementarity on the flagged set	56
C	Reproducibility	59

1 Introduction and Context

Intrusion detection is an operational problem as well as a classification problem. Missed attacks can leave harmful activity unchallenged, while false alarms consume analyst attention and make genuine incidents harder to prioritise. Practical guidance therefore treats alert validation, false-positive identification, and threat prioritisation as central parts of intrusion monitoring [19].

Networks produce a continuous stream of connections whose meaning is rarely contained in a single record. A destination port, a packet count, or a flow duration can be informative, but the surrounding communication pattern often matters just as much. A distributed denial-of-service attack may appear as many sources converging on one destination, whereas a scan may appear as one source contacting many hosts or services. This relational character makes network traffic a natural candidate for graph-based analysis. Network events also carry semantic cues: protocol and port combinations, traffic magnitudes, and recurring behavioural descriptions can suggest what an interaction represents. An intrusion detection system that sees only one of these views is working with an incomplete account of the traffic.

This report investigates whether structural and semantic evidence can be combined in a single edge-level intrusion detection framework using NF-UNSW-NB15 and NF-ToN-IoT. One experimental boundary must be stated from the outset: the current graph construction aggregates flows by source IP, destination IP, and attack class. Ground-truth labels therefore participate in defining edge identity, even though the attack name is excluded from the model’s input features and generated text. The resulting scores compare representations and interaction mechanisms under a controlled protocol, but they are not deployment-valid estimates of detection performance on unseen, unlabelled traffic.

NF-UNSW-NB15 derives from UNSW-NB15, whose original benchmark was created from a mixture of normal activity and contemporary attacks in the UNSW Canberra Cyber Range Laboratory [16]. NF-ToN-IoT derives from the TON_IoT datasets and provides an IoT-focused telemetry setting [15]. This study uses both in their NetFlow-standardised forms [18]. One identical processing and modelling pipeline is applied to the two datasets. Every architectural choice is shared between them. Only two quantities selected on validation data differ.

The two datasets were chosen because they occupy very different points on the class-imbalance axis. After aggregation, NF-UNSW-NB15 contains 656 edges, is 47.4% benign, and has a most-to-least-frequent class ratio of 12.4:1. Eight of its nine attack classes contain 40 edges and the ninth contains 25, so just one falls at or below that threshold. NF-ToN-IoT contains 2,127 edges, is 82.1% benign, and has a most-to-least-

frequent class ratio of 582:1. Seven of its nine attack classes contain 35 edges or fewer. Its distribution is far more skewed. By the most-to-least class ratio, the two graphs differ by roughly a factor of 47 in imbalance severity.

That difference is the methodological reason for using two datasets. A single dataset cannot separate a property of the architecture from a property of the class distribution that architecture happened to face. Holding the architecture fixed while changing the imbalance regime by roughly $47\times$ makes that distinction experimentally addressable. RQ4 already asks how class imbalance affects fusion and feedback, and the two-dataset design gives that question two substantially different regimes to compare. No stage is assumed in advance to be sensitive or insensitive to that change.

The scoring protocol also differs in one necessary respect. Macro-F1 covers ten classes on NF-UNSW-NB15 and eight classes on NF-ToN-IoT, because two NF-ToN-IoT classes are too small to survive five-fold splitting. Their edges remain in the graph for message passing but are excluded from the metric. Because the class counts differ, absolute scores are not comparable across the two datasets. Only the shape of the model ladder is. This constraint applies throughout the report.

1.1 Why combine graph and language representations?

Graph neural networks (GNNs) provide a direct way to represent communication structure. In this work, IP addresses form the nodes of a directed graph and aggregated flows form labelled edges. A graph-attention encoder propagates information between communicating hosts and produces a structural representation for each edge. This branch can learn patterns that are difficult to express as independent rows in a conventional feature table. It does not, however, possess an explicit vocabulary for interpreting the traffic. The graph tells the model *who communicated with whom* and how that interaction is positioned in the network. It does not explain what a high-frequency TCP connection to a particular service may signify.

A pretrained cybersecurity language encoder offers a complementary view. The same aggregated flow can be written as a short, controlled sentence describing its endpoints, protocol, port, frequency, volume, and duration. The implementation uses CySecBERT, a BERT-class model adapted to cybersecurity text and evaluated by its authors on domain-specific representation and classification tasks [2]. It is frozen and used purely as a sentence encoder. Throughout this report, this component is called the *semantic branch*. That terminology distinguishes its representational role from that of a generative large language model.

Neither representation is assumed to win in every case; the design instead lets them challenge and complement one another. The parallel fusion stage runs the graph and

semantic encoders side by side and combines their edge embeddings through Adaptive Gated Attention Fusion (AGAF). The feedback stage adds uncertainty-guided semantic feedback: edges with high-entropy GNN predictions are selected for semantic consultation, and the resulting signal is reintroduced by modifying the edge representations that the next graph update consumes. A second design, which biases graph attention directly, was also implemented and measured. It produced no change in prediction at all, and is reported as a verified negative result.

The coupling is bidirectional in routing. The graph model’s per-edge uncertainty decides which edges are consulted, while the semantic branch returns a per-edge judgement that re-enters the edge representations consumed by the next graph update. It is not bidirectional in content: the consultant’s judgement depends on that edge’s text alone, not on the graph model’s current belief about the edge.

This general dual-stream idea is not novel by itself. Parallel BERT and graph encoders with attention-based fusion have already been studied outside network security, for example in misinformation detection [14]. Work in network intrusion detection has also combined GNNs and generative language models for a different role: XG-NID performs intrusion classification with a heterogeneous GNN and uses an LLM after classification to generate explanations and remedial guidance [5]. DAS is the closest prior work to the feedback mechanism considered here. It already implements a closed feedback loop between a frozen GNN and an LLM, feeds refined semantics back to update the same graph learner, and constructs node descriptions by expressing structural statistics in natural language. Its setting is node-level and concerns general text-rich graphs [21]. The contribution here is therefore instantiation plus measurement: an edge-level network-intrusion-detection instantiation, together with a diagnostic question about whether a constrained semantic consultant exploits a feedback channel that has the capacity to influence the next graph update. Within the literature reviewed here, we are not aware of prior work combining bidirectional iterative coupling between a graph model and a semantic branch, joint attention-based fusion for the final classification, and edge-level network intrusion detection.

1.2 Research problem and questions

The research problem is not simply whether adding a language embedding raises a score. If one branch is already a strong classifier, a fusion module can appear successful while doing little more than copying that branch. Likewise, a feedback loop can improve the final output because semantic features are fused at the classifier, even when the claimed attention re-entry mechanism contributes nothing. The evaluation must separate the value of the two modalities, the fusion mechanism, the uncertainty-selection policy, and

the iterative update itself.

Five research questions frame the study:

- RQ1.** How do the structural GNN branch and the semantic branch perform individually on NF-UNSW-NB15 and NF-ToN-IoT under the same five-fold out-of-fold evaluation protocol?
- RQ2.** Does AGAF score above both single-branch rungs on NF-UNSW-NB15 and NF-ToN-IoT, and are those differences separated under the bootstrap protocol?
- RQ3.** Does uncertainty-guided semantic feedback add value beyond static fusion on NF-UNSW-NB15 and NF-ToN-IoT? If so, does the gain come from semantic re-entry into the graph representation or from final-output fusion?
- RQ4.** How do graph topology, repeated endpoint pairs, class imbalance, and the strength of the semantic consultant affect fusion and feedback on NF-UNSW-NB15 and NF-ToN-IoT?
- RQ5.** How does the feedback loop compare against published graph-based intrusion detection architectures re-trained on the same aggregated representation under the same protocol on NF-UNSW-NB15 and NF-ToN-IoT, and how much of any observed difference is attributable to representation and tuning rather than to architecture?

These questions deliberately allow a negative answer. The experiments are meant to determine when the proposed interaction works, when it does not, and why. A model ladder of GNN, semantic encoder, fusion and then feedback is only meaningful if each higher rung is compared fairly with the strongest relevant lower rung.

1.3 Objectives and contributions

The practical objective is to build one reproducible pipeline that transforms NF-UNSW-NB15 and NF-ToN-IoT NetFlow records into two aligned views and evaluates every stage under one consistent protocol. The work makes five contributions.

1. It implements a directed, edge-classification graph pipeline in which flows are aggregated by source IP, destination IP, and attack class, with centrality-based node features and statistical edge attributes.
2. It constructs a corresponding internal knowledge representation and serialises each edge as controlled natural language. The text supplied to the encoder is

kept label-free by an explicit validation guard, preventing the semantic branch from seeing the target attack name or a synonymous relation label.

3. It implements and compares two multimodal stages: AGAF for parallel structural-semantic fusion, and uncertainty-guided semantic feedback that reintroduces semantic evidence into the edge representations of uncertain edges. The feedback loop consults a classification head trained on the semantic embeddings. The loop is reported under that consultant and under the weaker embedding-only scorer it was first built with, so the reader can see how much of the result is the mechanism and how much is the consultant.
4. It provides a controlled evaluation on NF-UNSW-NB15 and NF-ToN-IoT, including unimodal baselines, fusion comparisons, uncertainty-selection ablations, trained-head baselines, mechanism-isolation tests, and two published graph-based intrusion detection architectures re-trained on the same representation. It reports both positive and negative results to show which components carry the measured gain, including five measured attempts to make an embedding-only consultant’s advice usable by the feedback channel, none of which succeeded.
5. It provides a cross-dataset comparison under one shared architecture across two substantially different class-imbalance regimes, testing whether the behaviour of each stage in the model ladder is a property of the method or of the class distribution it faces.

The contribution is not presented as a universal improvement over existing intrusion detectors. It is an implementation and controlled evaluation of a particular form of structural-semantic cooperation, together with evidence about its limits. The headline of that evidence is stated here so it is not mistaken for a footnote later. With a trained semantic consultant the feedback loop is the highest rung on both datasets, separated from the graph encoder, from AGAF and from every faithful re-trained baseline. With the weaker consultant it was first built with, none of those comparisons is separated at all. And when final-output fusion is removed, so that semantic evidence can reach the classifier only by re-entering the graph, neither consultant produces a gain we could separate from zero. The channel is live and measurable, and what the re-entry itself adds is, on these graphs, nothing we could distinguish from nothing. Deployment-valid evaluation remains open and requires graph construction in which edge identity can be determined without access to the target class.

1.4 Report structure

The report is organised into five sections. This Introduction defines the problem, experimental scope, research questions, and contribution. Section 2 reviews graph-based intrusion detection, cybersecurity language encoders, structural–semantic fusion, and iterative graph–language feedback, and positions the study against the closest prior mechanisms. Section 3 describes the two datasets, graph and text construction, the structural and semantic branches, AGAF, uncertainty-guided semantic feedback, the baseline protocol, and the shared evaluation procedure.

The Results section reports the predefined model ladder, baseline comparisons, ablations, and mechanism-isolation measurements for both NF-UNSW-NB15 and NF-ToN-IoT, while keeping the different scored class sets explicit. The Discussion section interprets those measurements against the research questions, including the cross-dataset imbalance comparison and the distinction between a feedback channel having capacity and a constrained semantic consultant making use of that channel. It also returns to the controlled-protocol limitation and the work still required for deployment-valid intrusion detection.

2 Related Work

2.1 Graph-based network intrusion detection

Intrusion detection is conventionally divided into signature-based and anomaly-based approaches [19], but either can be posed over a flow graph. A flow graph makes the edge the natural prediction unit when the object to be classified is a communication event between two endpoints. Nodes represent communicating entities such as IP addresses, and edges represent flows or aggregated flows. This formulation preserves the relational context around a connection without changing the classification target: the decision remains edge-level. It also separates information attached to the communicating endpoints from information carried by the flow itself.

E-GraphSAGE is an important reference point for this formulation. It applies a GNN to intrusion detection and classifies an edge from the concatenation of the embeddings of its two endpoints [11]. The edge’s own features do not reach the classifier. That choice gives the classification head a purely endpoint-derived view of the connection. A direct consequence is that two parallel edges with the same ordered endpoints present the same pair of endpoint embeddings to the edge classifier, even if the edges differ in attributes that are not otherwise represented at the head. The architecture can express structural context around an endpoint pair, but its final edge representation does not preserve an explicit channel for the attributes of the particular edge being classified.

TE-G-SAGE moves to an edge-aware GraphSAGE formulation: its supervised edge head consumes the edge’s own features, and the model is paired with SHAP-based explanation [13]. This changes the representational question. Once the edge itself is allowed to carry predictive information, the model can distinguish endpoint context from flow-specific attributes rather than requiring the classifier to infer the latter indirectly from node embeddings.

For network-flow intrusion detection that distinction matters, because the graph and the individual connection encode different kinds of evidence. The graph describes who communicates with whom and under what relational context. The edge describes the flow being classified. The graph-based IDS literature therefore motivates an edge-level representation retaining both endpoint structure and edge-specific content. The next question is whether edge content has to remain numeric, or whether some of it can be represented semantically.

2.2 Language models over security text and network traffic

Cybersecurity language encoders provide one route to domain-specific semantics. Cy-SecBERT is a BERT-class encoder adapted to cybersecurity text [2]. Its relevance here is representational: a pretrained cybersecurity language encoder maps controlled security-related text into a dense embedding without requiring the graph encoder to interpret textual tokens itself.

A separate line of work treats traffic as a language-like sequence. BSTFNet applies a BERT-class model, ET-BERT, to serialised packet-level traffic and combines that representation with a spatiotemporal branch for encrypted-traffic classification [7]. It uses no graph encoder. The precedent that matters is the serialisation step itself: network observations that begin as structured traffic fields can be converted into a controlled token sequence and processed by a BERT-class encoder.

These two lines solve different problems. Domain adaptation addresses the vocabulary and semantics of cybersecurity text. Traffic serialisation creates text-like input from network measurements. Together they support a semantic branch that encodes a flow description as text rather than treating every field as a scalar magnitude. They do not, by themselves, determine how that semantic representation should interact with graph structure.

That interaction is the central fusion problem. Once a system holds one representation derived from topology and another derived from text, the design space is no longer only about which encoder to choose. It is about where the two modalities meet, and whether their relative influence is fixed across samples or allowed to change from one sample to the next.

2.3 Fusing structural and semantic representations

Fusion methods organise along two related axes: how tightly the modalities interact, and whether the fusion rule assigns the same modality weighting to every sample or adapts that weighting to the input. The second axis is the more important one here. Four recurring families cover the relevant design space.

The simplest family is late fusion with a fixed interface between branches. GMLM uses separate graph and language towers, concatenates their representations, and passes the result to an MLP [20]. Its own comparison explicitly distinguishes this design from cross-attention, gating, and learned modality weights. BertGCN gives an even cleaner example of fixed weighting: its final prediction linearly interpolates a GCN prediction and a BERT prediction using a global hyperparameter [10]. Every sample receives the same interpolation coefficient. These methods keep the encoders modular, but the

fusion rule never decides that one modality should matter more for one input and less for another.

A second family introduces input-dependent gates. The Gated Multimodal Unit uses multiplicative gates to determine how each modality contributes to the fused activation [1]. This is the canonical primitive for sample-adaptive modality weighting. A graph-and-text instance appears in the Dual-Stream Graph-Augmented Transformer for fake-news detection, where BERT runs in parallel with a GAT and Graph Transformer branch, and a trainable scoring function emits a scalar weight for each modality for each input [14]. Mechanically this is the closest analogue in the literature surveyed here to a fusion stage whose balance changes per sample.

A third family uses attention to let the modality representations interact directly. RAGFormer stacks the two branch embeddings as a short token sequence and applies self-attention over them with a residual connection [9]. CAST uses cross-attention between graph and text representations in materials prediction and motivates the text branch by arguing that graph conversion can discard global information which textual descriptions restore [8]. The semantic representation is therefore not a duplicate encoding of graph structure. BiGTex goes further and interleaves a two-way exchange: structural information enters the language model as a soft prompt, and text returns to the graph side through cross-attention with the query supplied from the graph side [3]. Attention-based interaction differs from global interpolation because its interaction weights are derived from the representations of the current input.

The fourth family removes a distinct late-fusion stage altogether by interleaving structural and semantic processing throughout the encoder. DET uses exactly the terms “structural encoder” and “semantic encoder” and argues against late fusion of separate encoders, preferring interleaved interaction [6]. GL-Fusion sits at the opposite end of the coupling spectrum from independent towers: message passing is folded into every transformer layer, leaving no separate fusion module [24]. These designs mix the modalities repeatedly instead of preserving two independent representations until classification.

The taxonomy separates two questions that are often collapsed. One is where graph and text meet: after separate encoding, inside a dedicated attention mechanism, or repeatedly throughout the encoder. The other is whether the relative influence of the modalities is globally fixed or varies with the sample. The second distinction becomes sharper on moving from general graph-text systems to intrusion detection, where the available GNN-language combinations tend to assign the two components sequential roles rather than fusing their representations.

2.4 GNN and LLM combinations in intrusion detection

Within the intrusion-detection works surveyed here, graph models and language models are combined sequentially rather than fused as two representations of the same edge-level decision. XG-NID is the clearest example. Its heterogeneous GNN performs the intrusion classification, and an LLM is used only after prediction, to produce human-readable explanations [5]. The system’s “dual modality” refers to flow-level and packet-level data inside the graph pipeline, not to graph and text embeddings fused for classification.

Agent-based analysis for GNN robustness reverses the order. descriptions of node interactions and flags, removes, or reweights suspicious nodes and edges before the GNN processes the graph [25]. The language component acts as a preprocessing filter: its output changes the graph presented to the GNN, but the two components are never combined as concurrent representations at the classifier.

Other intrusion-detection systems occupy adjacent points in the design space without forming a GNN–language pair at all. GCN-2-Former combines a GCN with a temporal transformer and fuses the two by global average pooling and concatenation [26]. The second tower is a temporal transformer, not a language model. CPS-IDS uses gated fusion between a DeBERTa payload-semantic branch and a statistical branch [23], so it has both a language-based semantic component and an explicit fusion mechanism, but no graph encoder.

The distinction is functional rather than terminological. In this literature the language model either explains a GNN output after classification or alters graph inputs before message passing. The parallel-fusion examples occupy nearby configurations but either replace the language model with a non-language temporal transformer or replace the graph encoder with a statistical branch. Representation-level fusion and graph–language feedback therefore have to be examined in the broader graph-learning literature rather than inferred from the intrusion-detection examples alone.

2.5 Iterative GNN–LLM feedback

The broader GNN–LLM literature already contains the feedback mechanisms this report builds on. They are prior work and are not claimed here as contributions.

DAS is the closest prior work to the feedback mechanism used in this report. It implements an explicit closed feedback loop between a frozen GNN and an LLM: GNN predictions provide implicit supervision that steers semantic refinement, and the refined semantics are fed back to update the same graph learner [21]. DAS also constructs node descriptions by expressing structural statistics in natural language, using degree,

betweenness, closeness, clustering coefficient and square clustering coefficient, and gives a Majorization–Minimization argument under which its refinement loop monotonically decreases a task-adaptive surrogate objective. Its setting is node-level, over general text-rich graphs. Closed graph–language feedback, and the conversion of structural statistics into natural-language descriptions, already exist in the literature.

LOGIN already couples uncertainty selection to language-model consultation. It uses MC-dropout variance to identify uncertain nodes, prompts the LLM with node text and one-hop topology, and receives a predicted label together with a free-text explanation [17]. Incorporation depends on correctness: if the LLM’s label matches the ground-truth label, the explanation is appended to the node text and the augmented text is re-embedded. If the label is wrong, edges in the node’s one-hop neighbourhood are pruned. The LLM remains outside autograd and edits the learning inputs between training rounds. Selecting uncertain graph elements for semantic consultation is also established.

GLANCE makes the decision to consult the language model itself learnable. A lightweight router decides per node whether querying is worthwhile from inexpensive node features, and an advantage-based objective trains that router on the outcome of querying versus not querying, because the LLM call is non-differentiable [12]. Its gains concentrate on heterophilous and structurally ambiguous nodes. This addresses the same allocation problem as uncertainty-based consultation, directing language-model effort at selected graph elements rather than applying it uniformly, with a learned criterion in place of a fixed uncertainty rule.

RoGRAD places iterative refinement, rather than static one-shot signal injection, at the centre of its formulation [22]. It studies that refinement under graph deficiencies including noise, sparsity, and label scarcity. Its abstract claims priority for moving language-model augmentation of graphs from static injection to an iterative refinement paradigm. Iterative semantic refinement is already an explicit contribution claim in the prior literature.

These four anchors settle the component question plainly. Closed GNN–language feedback already exists. Iterative semantic refinement already exists. Structural statistics have already been expressed as natural-language input. Uncertainty and learned routing have already been used to decide which graph elements receive language-model consultation. Taken component by component, the feedback mechanism is not the contribution of this report.

2.6 Positioning

The contribution is an edge-level intrusion-detection instantiation, and a measurement question about how such a feedback channel is used. The comparison has to be made at the level of configuration, not of individual mechanisms.

The four feedback anchors operate at node level, whereas the present setting treats a network flow as an edge and makes the classification decision there. Their semantic consultants are prompted generative LLMs. Here the semantic branch is a frozen sentence encoder whose embeddings are scored against class prototypes. Their feedback actions refine semantic inputs or edit graph inputs. Here the accepted semantic signal modifies the edge representations consumed by the next graph update. These are differences of instantiation, not claims that the underlying ideas originated here [12, 17, 21, 22].

Two signals travel in opposite directions in the resulting loop. The graph model’s per-edge uncertainty determines *which* edges are routed to the semantic branch, and that selection is recomputed as predictions shift. The semantic branch returns a per-edge judgement that re-enters the graph model’s edge representations. The coupling is bidirectional in this routing sense. It is not bidirectional in content: the consultant’s verdict on an edge depends on that edge’s text alone, not on what the graph model currently believes about it.

The final classification stage adds a further dimension to the configuration. Structural and semantic edge representations are combined through joint attention-based fusion with sample-dependent modality weighting, rather than assigning the semantic component a preprocessing or post-hoc role. Sample-adaptive fusion is itself established outside this setting: gated multimodal units provide input-dependent modality control, and the Dual-Stream Graph-Augmented Transformer assigns per-input scalar modality weights to parallel text and graph representations [1, 14]. Representation-level graph-text interaction is likewise established, through self-attention, cross-attention, and interleaved exchange [3, 8, 9]. None of those mechanisms is a contribution claim here.

Within the limits of the literature search represented by this survey, we are not aware of prior work combining all three of the following in one system: bidirectional iterative coupling between a graph model and a semantic branch, joint attention-based fusion for the final classification, and edge-level network intrusion detection. This is a statement about the configuration space our search covered. It does not imply a performance advantage, and it does not change the prior-art status of any individual component.

That positioning leaves a narrower research question. Existing feedback work asks whether a language-model-driven refinement process improves the graph-learning task, with the model revising semantics, modifying inputs, or selectively augmenting graph

elements [12, 17, 21, 22]. The present work instead instantiates the loop in network-flow intrusion detection with a deliberately more constrained consultant, and asks a diagnostic question: whether such a consultant exploits a feedback channel that has the capacity to influence subsequent graph updates. That question is underexplored in the work surveyed here. No claim about its answer belongs in this section.

The contribution is best described as instantiation plus measurement. The system instantiates bidirectional feedback and sample-adaptive structural–semantic fusion for edge-level network intrusion detection, then measures whether the constrained semantic consultant makes use of the channel it is given. It does not rest on inventing GNN–LLM feedback loops, iterative semantic refinement, natural-language descriptions of graph statistics, or selective semantic consultation. Those mechanisms have clear precedents. The point of this configuration is to make their interaction measurable in an edge-level network-IDS setting.

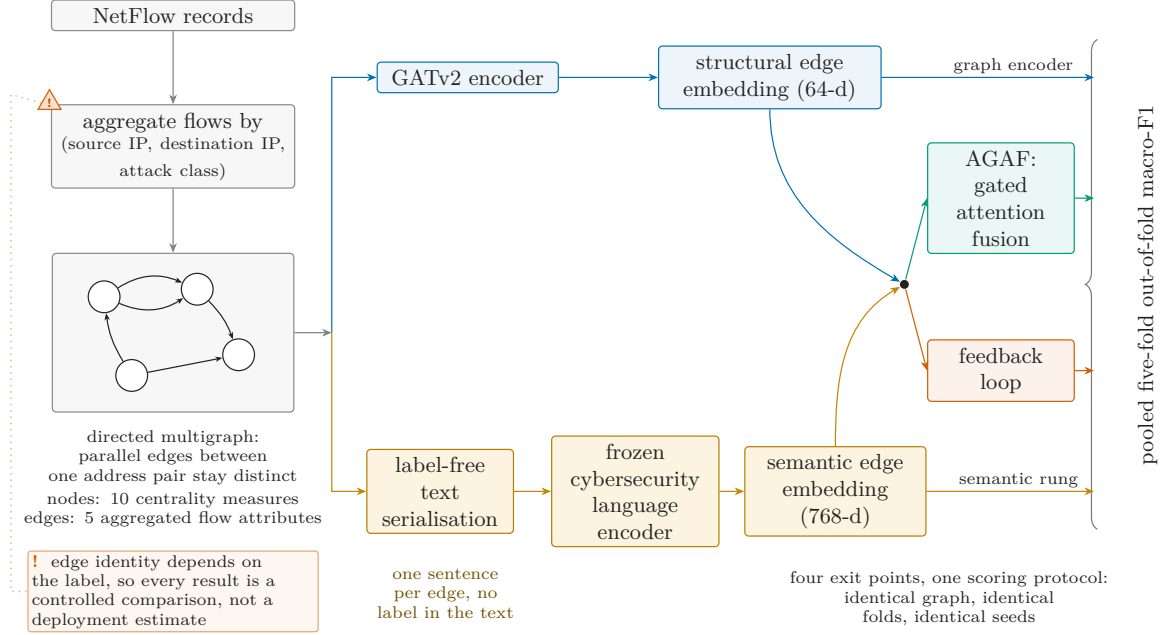


Figure 1: The pipeline, from flow records to four scored rungs. Blue is the structural branch, orange the semantic branch, green the fusion stage and vermillion the feedback loop. Every rung is an exit point from one shared graph and is scored by one protocol on one fold partition and one set of seeds. The marked aggregation step is the label-conditioned grouping key that bounds every claim in this report.

3 Method

3.1 Datasets and Common Processing Pipeline

We evaluate on NF-UNSW-NB15 and NF-ToN-IoT, the NetFlow-standardised variants of their respective intrusion-detection corpora [16, 18]. One pipeline processes both, and every architectural choice is shared between them. Only two quantities are permitted to differ, and both are selected on validation folds: the uncertainty threshold k and the feedback injection scale. This restriction is deliberate. A cross-dataset comparison in which the two ladders use different components measures the components, not the datasets.

The pipeline converts flow records into a graph, derives node and edge attributes, trains an edge-level structural model, builds label-free semantic representations for the same edges, and combines the two modalities through adaptive fusion. Every classification decision is edge-level: a flow, or an aggregate of flows sharing a grouping key, is an edge, never a node. Figure 1 shows the whole pipeline and the four points at which it is scored.

3.2 Directed Multigraph Construction

Flows are aggregated into a directed multigraph keyed on

(source IP, destination IP, attack type).

Parallel edges therefore exist between an ordered IP pair whenever those flows carry different attack types. Nodes are the distinct IP addresses. The resulting graphs are small: NF-UNSW-NB15 yields 49 nodes and 656 edges, NF-ToN-IoT 1,501 nodes and 2,127 edges.

Each node carries ten centrality columns: betweenness, PageRank, degree, closeness, eigenvector, k -core, k -truss, global betweenness, global PageRank, and modularity vitality. These are computed on an unweighted IP-to-IP graph with self-loops removed. Betweenness (normalised), PageRank, closeness and eigenvector centrality are computed on the directed graph. Degree is *in-degree* centrality, and k -core and k -truss are computed on the undirected projection. Modularity vitality is the leave-one-node-out drop in modularity under a greedy modularity partition of the undirected projection. Eigenvector centrality and the truss decomposition fall back to constants when they fail to converge. A node’s feature vector is the mean of these values over all incident edges, taking the node in both its source and destination roles.

One property of this feature set should be stated outright. On NF-UNSW-NB15 the “global” variants are populated from the same computation as their local counterparts, so `global_betweenness` and `global_pagerank` are exact duplicates of `betweenness` and `pagerank`. The ten columns span eight distinct measures. On NF-ToN-IoT, whose centralities were computed upstream, the global variants differ from the local ones. We report ten input channels because that is what the model receives, but the effective rank of the node features is lower on NF-UNSW-NB15.

Each edge carries five attributes. Three are numeric: the number of constituent flows (`flow_count`), their summed byte count (`total_bytes`), and their mean duration (`avg_duration`). Two are categorical: the modal protocol and the modal port, with ties broken toward the smaller value.

Because attack type is part of the aggregation key, the label participates in defining edge identity. This is a property of the protocol, not a bookkeeping detail. Two flows between the same IP pair that differ only in their label become two distinct edges, and a deployed system observing unlabelled traffic could not perform this separation. The scores reported in this work therefore compare representations and mechanisms under a controlled protocol. They are not deployment-valid detection estimates, and we do not present them as such. Figure 2 gives the resulting class sizes on both graphs.

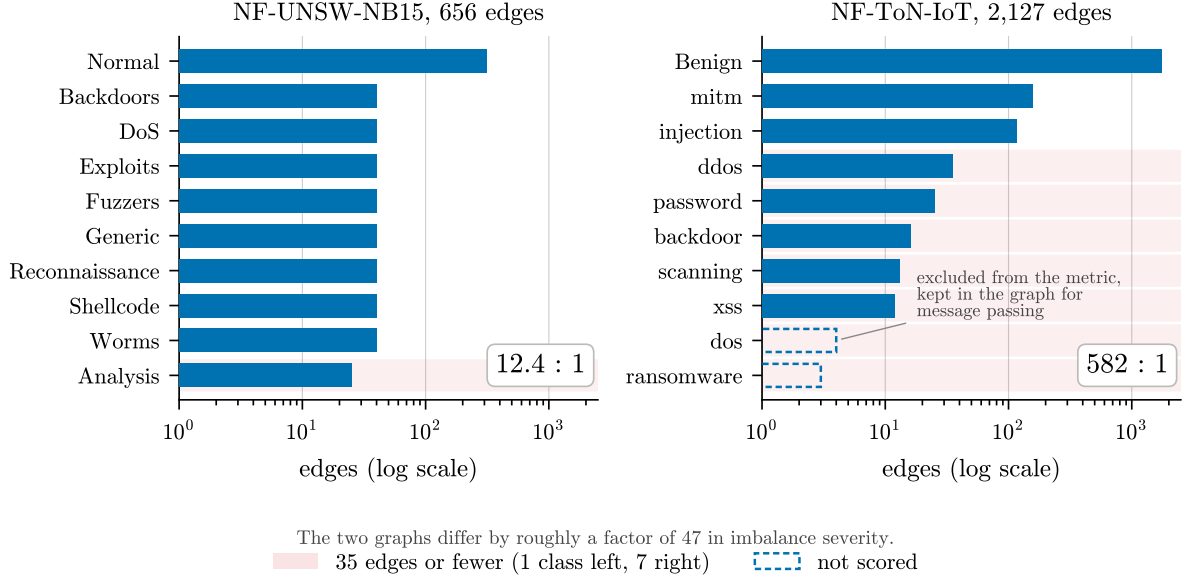


Figure 2: Class sizes on a logarithmic axis, largest first. NF-UNSW-NB15 has one class at or below 35 edges; NF-ToN-IoT has seven. The two smallest NF-ToN-IoT classes stay in the graph for message passing and are excluded from the metric. The two graphs differ by roughly a factor of 47 in the ratio of the largest class to the smallest scored class.

3.3 Edge Feature Encoding

The encoding separates quantities from identifiers. Columns 0–2 receive a $\log(1 + x)$ transform, then Z-scoring. Columns 3–4 are rewritten as contiguous vocabulary indices and consumed by learned embedding tables of width 4 (protocol) and 32 (port). Concatenation gives a $3 + 4 + 32 = 39$ -dimensional encoded edge vector. The vocabularies are small: 7 protocols and 169 ports on NF-UNSW-NB15, 4 and 235 on NF-ToN-IoT.

This corrects a specific defect in an earlier version of the pipeline, which passed protocol and port through as raw magnitudes. Port then entered the network at roughly $11,000\times$ the scale of every normalised column, and the ordering it imposed is meaningless: port 443 is not “more” than port 80. The embedding tables let the model represent protocol and port identity without inheriting the arithmetic of their indices.

The Z-score statistics and the two vocabularies are fitted once over all edges, before the cross-validation split. This is transductive, and we flag it explicitly. It is standard for a single fixed graph, and it transfers no label information, but it is not the same as fitting scalers inside each training fold.

Table 1: Per-variant edge focus weights. Applied as a fixed diagonal scaling of the encoded edge vector. The protocol and port weights are broadcast across their embedding blocks.

Variant	flow_count	total_bytes	avg_duration	protocol	port
temporal	1.25	0.75	1.50	0.50	0.25
content	0.50	1.50	0.50	1.00	1.25
behavioural	1.25	0.75	1.00	0.75	1.50

3.4 Structural Branch

Each structural encoder is a two-layer GATv2 network [4]. The first layer uses eight heads of width $64/8 = 8$, concatenated to 64. The second uses a single head of width 64. Both layers receive the 39-dimensional encoded edge vector as an attention input and add no self-loops. Each layer applies, in order, the GATv2 convolution, batch normalisation, an ELU nonlinearity, and a residual connection. Dropout at $p = 0.2$ is applied before the residual in the first layer only. The first layer’s residual passes through a linear projection from the 10 input channels to 64, and the second’s is an identity. The final node embedding is 64-dimensional.

We use GATv2 rather than GAT because of the static-attention limitation [4]: in GAT the ranking a node induces over its neighbours can be independent of the query node, so attention degenerates into a global neighbour scoring. GATv2 applies its attention MLP after the concatenation of both endpoints, which restores query-dependent ranking. On a graph with 49 nodes and heavy parallel-edge structure, that distinction is not cosmetic.

Three encoders run in parallel: *temporal*, *content* and *behavioural*. They are architecturally identical and differ only in a fixed diagonal reweighting of the encoded edge vector, given in Table 1. The protocol weight is broadcast across all 4 protocol-embedding dimensions and the port weight across all 32, so each variant keeps the same relative emphasis it would have had on the five raw columns. The reweighted edge vector is used both inside the attention computation and in the edge representation that follows, so a variant’s emphasis shapes its message passing as well as its readout.

Within a variant, the representation of a directed edge $e = (u, v)$ concatenates the two endpoint embeddings, their elementwise product, their elementwise absolute difference, and the variant’s reweighted edge vector $\tilde{\mathbf{x}}_e$:

$$\mathbf{z}_e = [\mathbf{h}_u, \mathbf{h}_v, \mathbf{h}_u \odot \mathbf{h}_v, |\mathbf{h}_u - \mathbf{h}_v|, \tilde{\mathbf{x}}_e] \in \mathbb{R}^{4 \cdot 64 + 39} = \mathbb{R}^{295}.$$

The product and absolute-difference terms matter here. Endpoint concatenation

alone cannot separate parallel edges, and 58.7% of NF-UNSW-NB15 edges share an IP pair with at least one other edge. The elementwise terms are symmetric functions of the endpoints and do not fix this on their own, but $\tilde{\mathbf{x}}_e$ does, by carrying the edge’s own features into the classifier.

Each variant projects $\mathbf{z}_e$ to 64 dimensions through a linear layer and ReLU. The three projections are concatenated and passed through a linear layer to 64, ReLU, and dropout, yielding the fused 64-dimensional edge embedding consumed by every downstream stage. A final linear layer emits class logits. Each variant additionally carries its own classification head, a two-layer MLP over $\mathbf{z}_e$ with a 64-unit hidden layer, ReLU and dropout, whose loss is added to the objective with weight 0.30. The auxiliary term keeps each variant directly supervised rather than letting the fusion layer absorb all gradient.

3.5 Semantic Branch

The second modality describes each edge in controlled natural language. The continuous edge quantities are first discretised into five rank-based quantile bands, computed globally across all edges, and the band label rather than the raw number enters the sentence. A template then renders endpoints, connection frequency, transferred volume, per-flow byte magnitude, duration band, protocol name, and a coarse port role (web, remote-access, well-known, registered, or ephemeral) into one sentence per edge, of the form: “*Observed traffic from source ... to destination ... with ... connection frequency, transferring ... as ..., ..., over ..., to a ... port ...*”

Sentences are encoded by CySecBERT [2], a BERT-class encoder domain-adapted to cybersecurity text. It is frozen throughout. No gradient reaches it. Inputs are tokenised with padding and truncation at 128 tokens, and the final hidden states are mean-pooled with padding positions masked out, giving one 768-dimensional vector per edge. We use it strictly as a sentence encoder. It is not prompted, not asked to reason, and not generative. We avoid describing it as an LLM for that reason.

The embedding is consumed in two ways, and the distinction matters for everything that follows. The *embedding-only* route fits no classifier on it: a whitened prototype scorer (described under the feedback stage below) assigns each edge to the nearest class mean in a fold-local whitened space. This is the semantic rung of the ladder and the semantic input to AGAF, and it measures what the encoder alone knows. The *trained head* route fits, within each fold, a small classifier on the training-fold embeddings: a per-fold standard scaler, then a 768–128–128– C multilayer perceptron with ReLU and dropout 0.2, trained with the same class-weighted focal loss, optimiser and early-stopping rule as the structural branch. Each fold’s head is applied to every edge and

its logits are stored: on that fold’s training and validation edges the logits are in-fold, on its test edges they are out-of-fold, so the pooled test score of the head is a legitimate out-of-fold number. That head is the consultant the feedback loop uses. Evaluated on its own, it is also the strongest single-modality baseline in this study. We report it in every table for that reason. A cross-fitted variant, in which training-edge logits come from an inner cross-validation so that the loop trains on advice no more reliable than the advice it is scored with, was implemented and measured. It scored lower on both datasets and is reported in Section 4. The in-fold head is retained.

Label freedom is enforced in code, not by convention. Before any sentence reaches the encoder, an assertion scans every generated sentence for every attack term present in the dataset and raises on the first match, naming the offending index and sentence. The pipeline halts. This is what makes the semantic branch an independent view of the edge rather than a relabelling of it.

3.6 Adaptive Gated Attention Fusion

AGAF projects each modality into a shared 128-dimensional space through a linear layer, batch normalisation and ReLU, so 64 to 128 for the structural embedding and 768 to 128 for the semantic one. A gate consumes the concatenated projections and emits two logits, softmaxed at temperature 1.0 into weights (α_g, α_l) that convexly combine them:

$$\mathbf{f}_e = \alpha_g(e) \mathbf{g}_e + \alpha_l(e) \mathbf{l}_e, \quad \alpha_g(e) + \alpha_l(e) = 1.$$

The weights are per-edge, so the balance between structure and semantics is free to vary across the graph rather than being fixed at concatenation time. A two-layer MLP over $\mathbf{f}_e$ produces class logits. Both inputs are standardised before entering the module. The scalars are held in the trainer and persisted separately, so the module’s `state_dict` stays free of fitted statistics.

One constraint on this stage is load-bearing. AGAF consumes *per-fold out-of-fold* structural embeddings only. It never receives embeddings from a GNN fitted on all edges, because such a model has already seen the test edges’ labels, and those labels would reach the fusion classifier through its own input features. That is a leak no split downstream of it could detect. For each held-out fold, the structural embeddings supplied to AGAF come from a model that never saw that fold’s labels. Earlier runs in this project violated this rule, and the reproduction entry point now enforces it.

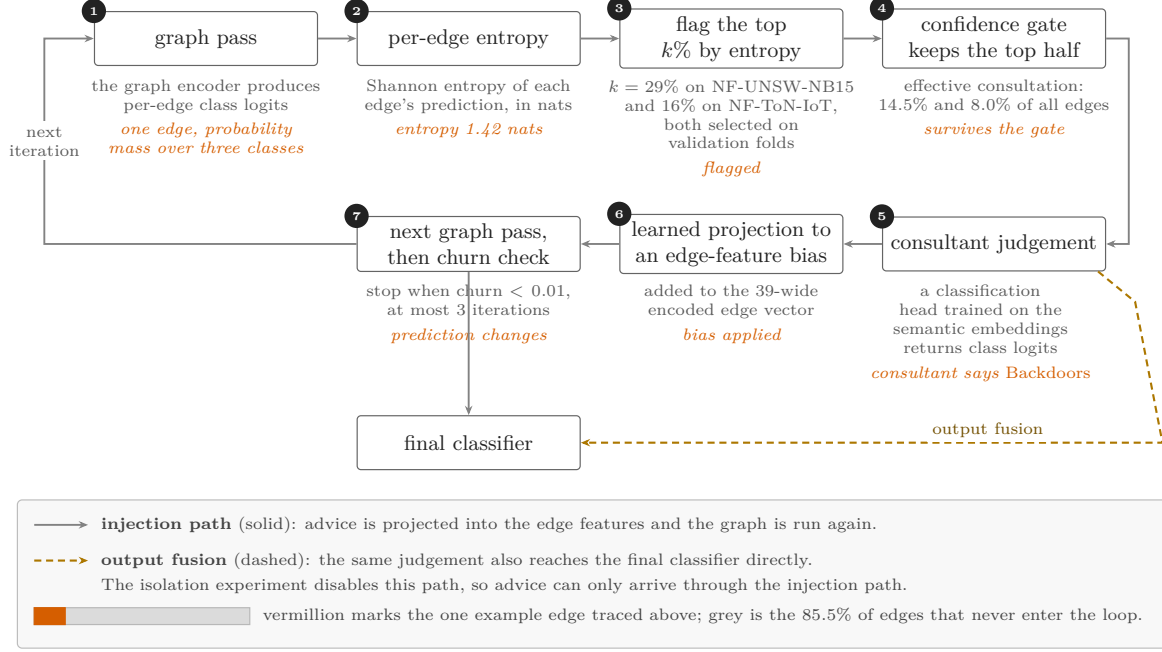


Figure 3: One pass of the feedback loop. The solid path is the injection mechanism under test: the consultant’s judgement is projected into the 39-wide encoded edge vector and the graph is run again. The dashed path is output fusion, which reaches the final classifier without passing through the graph, and which the isolation experiment of Section 4.3 disables. The percentages are the selected entropy thresholds and the effective consultation rates they produce.

3.7 Uncertainty-Guided Semantic Feedback

The feedback stage routes semantic information to the edges the structural model is least sure about, and only there. Figure 3 gives the stage as it runs.

Selection. For each edge we compute the Shannon entropy, in nats, of the GNN’s predictive distribution, and flag the edges above the $(1 - k/100)$ quantile. The threshold is calibrated once and reused, so the flagged set is a fixed-size top- $k\%$ rather than a moving target. We select k on validation folds only, over $[15, 35]$ at three training seeds: $k = 29.0$ on NF-UNSW-NB15 and $k = 16.0$ on NF-ToN-IoT. The validation curves are flat, with every candidate inside the noise band of the mean, so these values are the argmax of a flat curve rather than a tuned optimum, and we say so wherever they appear.

Consultation. The loop consults the per-fold trained head described in the semantic branch. For every flagged edge it reads that fold’s stored head logits, a C -dimensional class judgement computed from the edge’s sentence alone. The judgement does not depend on the graph model’s current belief about the edge, so the coupling is bidirectional

in routing but not in content.

The embedding-only alternative, used by the semantic rung and by AGAF, is a whitened prototype scorer. Within each fold we centre the training-fold embeddings, fit a ZCA whitener by eigendecomposition of their covariance with a floor on small eigenvalues (necessary because the training fold has fewer rows than the 768 embedding dimensions), and take the class-wise mean in the whitened space as that class’s prototype. An edge is whitened with the same fold-local transform and scored by cosine similarity against the prototypes. Whitening is not incidental: an earlier L2-normalise-average-renormalise scheme collapsed per-class spread until random and real advice became indistinguishable.

The rule that keeps the comparison fair applies everywhere: every rung uses the same encoder, the same graph, the same folds and the same training seeds. The loop additionally trains a classification head on the semantic embeddings, and that head is reported alone. The loop was first built with the prototype scorer as its consultant, on the argument that a consultant the loop already contains would only be echoed. That configuration, and five attempts to make its advice usable, are reported as negative results in Section 4. The trained head replaced it after those measurements.

Gating. Being uncertain in the structural view does not make an edge a good candidate for semantic advice. Among the flagged edges we keep only the most confident half by the consultant’s own maximum class probability, a semantic confidence fraction of 0.50. The effective feedback rate is thus 14.5% of edges on NF-UNSW-NB15 and 8.0% on NF-ToN-IoT.

Injection. Accepted advice is injected into the *edge representation* consumed by the next graph update. The consultant’s class logits are projected linearly to the 39-dimensional edge width, scaled by a learned strength, written into a full-graph bias tensor that is exactly zero on unflagged edges, and added to the reweighted edge vector:

$$\tilde{\mathbf{x}}_e \leftarrow \tilde{\mathbf{x}}_e + s \cdot \mathbf{b}_e, \quad \mathbf{b}_e = \mathbf{0} \text{ for unflagged } e,$$

with injection scale $s = 2.0$ on NF-UNSW-NB15 and $s = 20.0$ on NF-ToN-IoT, both selected on validation folds from $\{0.5, 1, 2, 5, 10, 20\}$ at three seeds. As with k , the curves are flat, and on NF-ToN-IoT the selected value sits on the upper boundary of the swept range, which we record. Because $\tilde{\mathbf{x}}_e$ enters both attention and readout, the injected signal propagates structurally rather than only shifting a final logit. The scale is re-selected whenever the edge encoding or the injection path changes, since it

is expressed in the units of that encoding.

Iteration. Predictions are recomputed, entropies re-derived, and the cycle repeats. We measure *churn*, the fraction of edges whose predicted class changed since the previous iteration, and stop when churn falls below 0.01 or after three iterations, whichever comes first.

Output fusion. The loop’s prediction is not the graph head’s logits after the last iteration. The refined edge embedding and the consultant’s logits are each projected to 128 dimensions and mixed by a per-edge gate $g_e \in (0, 1)$ that reads the entropy and maximum probability of both the graph head and the consultant, so it can route towards whichever view is confident on that edge. A two-layer classifier over the gated concatenation produces a correction, and the final logits are that correction plus the gated residual $(1 - g_e) \mathbf{z}_e^{\text{gnn}} + g_e \mathbf{z}_e^{\text{head}}$. The consultant’s own judgement is a floor the loop can fall back to on edges where the graph view is unsure. This stage has the same projection width as AGAF, so the loop and AGAF differ in where semantics enter (inside the graph update, or only at the output) and not in fusion capacity. Switching the stage off, together with the injection, yields the `head_only` control used throughout Section 4: the same network, trained in the same code path, with no semantic input at all.

An alternative that was implemented and measured. We also built a variant that adds the projected semantic signal to GATv2’s unnormalised attention logits, before the per-edge softmax, injecting into the attention mechanism rather than the edge representation. It is zero-initialised, so an untrained bias reproduces stock GATv2 exactly. It is evaluated under the identical protocol and reported in Section 4 as a negative result. We retain it in the paper because the two variants isolate *where* in the network a semantic signal must land to have any effect at all, a question the positive result alone does not answer.

3.8 Baseline Protocol

We compare against re-implementations of two published graph-based intrusion detectors, E-GraphSAGE [11] and TE-G-SAGE [13], retrained on the same aggregated edges, the same labels and the same folds. Each baseline keeps its own published featurisation rather than ours. E-GraphSAGE standard-scales all five NetFlow columns, TE-G-SAGE applies log1p, standard scaling, correlation pruning at 0.995, and one-hot categoricals with rare-category folding.

Each baseline is run in three configurations. `as_published` reproduces the released implementation. `refit` re-selects capacity and schedule on our validation folds only. `plus_node_features` is deliberately non-faithful: it hands the baseline our ten centrality node features, isolating how much of any gap is representation rather than architecture. We label it as non-faithful wherever it appears. It is not E-GraphSAGE and it is not TE-G-SAGE.

Five deviations from the published specifications were unavoidable at this graph scale, and we record each one. (D1) TE-G-SAGE’s chronological 60/30/10 split cannot be reconstructed, because aggregation collapses per-flow timestamps. Our stratified folds are used for all models. (D2) TE-G-SAGE’s neighbour fanout of (25, 15) and batch size of 4096 both exceed the entire graph, so training is full-neighbourhood and full-batch. (D3) Both released implementations train for a fixed epoch count with no validation split and no early stopping, 4,999 epochs for E-GraphSAGE and 20 for TE-G-SAGE, incompatible with a protocol that selects on validation folds. Baselines are given 300 epochs with patience 25 on validation macro-F1 and the best state restored. (D4) TE-G-SAGE’s `rare_min_freq` of 50 removes almost every port category at our scale. It is applied as published in `as_published` and swept in `refit`. (D5) E-GraphSAGE’s published Eq. 4 aggregates edge features alone, but the authors’ released code builds the message as $W_{\text{msg}}([\mathbf{h}_u \parallel \mathbf{e}_{uv}])$. We follow the code, since the published numbers came from it.

One design variable is internal to the loop rather than a rung of the ladder: the identity of the semantic consultant. The loop was first built around the whitened prototype scorer, the same one the semantic rung uses, and later around a per-fold classification head trained on the same frozen embeddings. Both versions are reported, at the same three seeds, at the same fold partition, and with the injection scale unchanged. Only the selected entropy percentile differs between them, at 31 against 29 on NF-UNSW-NB15 and 25 against 16 on NF-ToN-IoT.

Two asymmetries remain and should be weighed when reading the comparison. Our rungs train under a focal objective with patience 40, whereas the baselines use class-weighted cross-entropy with patience 25, each keeping its own published loss. And E-GraphSAGE classifies an edge from $[\mathbf{h}_u \parallel \mathbf{h}_v]$ alone, so parallel edges between one IP pair are mathematically indistinguishable to it. That covers 58.7% of NF-UNSW-NB15 edges and 7.9% of NF-ToN-IoT edges. That is an interaction between its architecture and our aggregated representation, not evidence that E-GraphSAGE is a weak detector, and we do not read it as one.

3.9 Evaluation and Training Protocol

We use stratified five-fold cross-validation over edges with partition seed 42, represented as boolean masks. Within each fold the held-out fifth is the test partition, and the remaining four-fifths are split 75/25 into training and validation with seed 42 + fold index, giving approximately 60/20/20. Test masks are used once, for prediction.

The primary metric is pooled out-of-fold macro-F1: each edge is predicted only while held out, and the five sets of out-of-fold predictions are pooled before a single macro-F1 is computed. Model selection uses validation folds exclusively. The uncertainty threshold k and the injection scale were selected per dataset by a validation sweep ($k \in [15, 35]$; scale $\in \{0.5, 1, 2, 5, 10, 20\}$) averaged across three seeds (42, 1, 2). The remaining architectural hyperparameters were fixed a priori and not swept. We note this as a limitation: k was selected on rotating validation folds rather than by nested cross-validation, so the final estimate is not fully unbiased with respect to that one choice.

Every rung is trained at three training seeds (42, 1, 2) with the fold partition held fixed at the seed-42 split, so all seeds score the same held-out edges and differ only in initialisation, dropout and data order. We report the mean and standard deviation of pooled out-of-fold macro-F1 over the three seeds. The standard deviation therefore measures training-seed variance at a fixed partition, not total uncertainty, since fold-partition variance is not measured. Pairwise differences between rungs use a two-level bootstrap with 2,000 iterations: each iteration draws a training seed independently for each side and resamples edges with replacement, so the interval reflects both sources of variation. A seed-matched bootstrap (seed 42 against 42, 1 against 1, 2 against 2) is reported as a secondary check. The decision rule was fixed before the runs: a difference is called *separated* only if its two-level 95% interval excludes zero, and *not separated* otherwise; we do not describe point-estimate orderings as significant. The semantic rung is deterministic given the folds and has zero seed variance, so it enters comparisons as a fixed side. Comparisons against re-trained published baselines use the same procedure with three baseline seeds on the other side.

Training uses AdamW with learning rate 10^{-3} and weight decay 5×10^{-4} , at most 300 epochs, early stopping with patience 40 on validation macro-F1 with the best state restored, and gradient clipping at norm 1.0. The objective is focal loss with $\gamma = 2.0$ and α set to inverse-frequency class weights computed on the training fold, normalised to sum to the class count, and classes absent from a training fold receive weight zero. Focal loss down-weights already-correct examples, which matters when rare attack classes hold 12–35 edges against a benign majority. We do not additionally oversample: combining oversampling with these weights double-corrects the imbalance

and collapses the fusion stage.

Every run exports `OMP_NUM_THREADS=1`. Graph-attention message passing sums neighbour contributions in thread-completion order, and floating-point addition is not associative, so thread count alone moves pooled macro-F1 by roughly 0.012 at a fixed seed, larger than several of the effects this paper reports. Semantic encoding is likewise pinned to a single device, because MPS and CPU produce slightly different embeddings and every downstream number inherits the difference.

Two NF-ToN-IoT classes are smaller than the five-fold minimum: `dos` with 4 edges and `ransomware` with 3. They remain in the graph and continue to participate in message passing, but are excluded from the reported metric, so NF-ToN-IoT macro-F1 is computed over 8 classes and NF-UNSW-NB15 over 10. The two are not comparable in absolute level, and we compare only the shape of the ladder across datasets, never the values.

4 Results

This section reports what was measured. Interpretation is deferred to Section 5. The wording is deliberately narrow: a difference is called *separated* only when its bootstrap interval excludes zero, and an ordering of means is called an ordering of means.

4.1 The protocol as it was run

Every rung is evaluated by pooled five-fold out-of-fold macro-F1. Each fold’s held-out predictions are collected, the five folds are concatenated into one prediction vector covering every edge exactly once, and macro-F1 is computed on that vector. The metric covers all ten classes on NF-UNSW-NB15 and eight of ten on NF-ToN-IoT, where **dos** (4 edges) and **ransomware** (3 edges) remain in the graph for message passing but cannot survive a five-way split. Because the class counts differ, the two datasets’ absolute levels are not comparable. Only the shape of the ladder is. Each rung was trained at three seeds, 42, 1 and 2, with the fold partition held fixed at the seed-42 stratified split. Only training stochasticity varies, so the pooled out-of-fold edge set is identical across seeds. That identity makes the seed-matched bootstrap a paired comparison. Reported standard deviations are training-seed variance at a fixed partition. Fold-partition variance was not measured, so every $\pm$ in this section is a lower bound on total uncertainty.

Two bootstrap procedures are used, both at 2,000 iterations. The primary is two-level: it resamples edges and independently draws a training seed, so seed-to-seed variation enters the interval. The secondary is seed-matched, pairing seed 42 with seed 42 and so on, resampling edges only. The seed-matched intervals are narrower by construction. They are reported so the reader can see how much of an interval’s width is seed variance. The decision rule was fixed before the runs: separated if the 95% two-level interval excludes zero, not separated otherwise. Where the two procedures disagree, the two-level result decides and both are shown.

The semantic rung is deterministic given the folds and the frozen embeddings. Its standard deviation is exactly 0.0000 at three seeds, and it enters every comparison as a fixed side. All runs export `OMP_NUM_THREADS=1`. That pin is not cosmetic: unpinned runs drift by roughly 0.012 macro-F1 at a fixed seed, larger than several of the differences reported below. One earlier finding in this project was retracted for exactly that reason. Re-running the graph and semantic rungs at seed 42 reproduces their committed values exactly. That has not always been true. A pre-v2 NF-ToN-IoT ladder failed to reproduce on two of four rungs in August 2026. Thread pinning became mandatory afterwards.

Finally, one boundary carries through everything below. Edges are aggregated by source IP, destination IP and attack class, so the label participates in defining edge identity. These are controlled comparisons between representations and mechanisms, not deployment-valid detection estimates.

4.2 The model ladder

Table 2 gives the four rungs on both datasets. The loop appears twice, because the identity of the consultant it queries is itself an experimental variable. The loop was first built around an embedding-only whitened prototype scorer, the same scorer that constitutes the semantic rung. It was later rebuilt around a per-fold classification head trained on the same frozen semantic embeddings. Both versions are reported throughout, since the change between them is the largest single effect measured in this work.

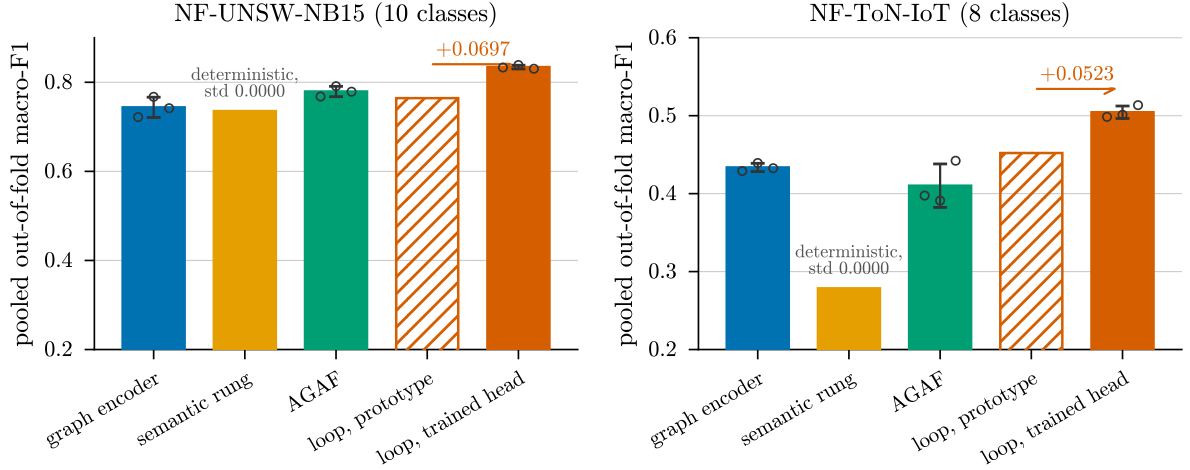
Table 2: Pooled out-of-fold macro-F1 by rung, three training seeds at a fixed fold partition. The loop is reported under both consultants. The $\pm$ is training-seed variance only. Absolute levels are not comparable across datasets.

Rung	NF-UNSW-NB15 (10 classes)			NF-ToN-IoT (8 classes)		
	mean $\pm$ std	seeds 42 / 1 / 2		mean $\pm$ std	seeds 42 / 1 / 2	
Graph encoder	0.7437 $\pm$ 0.0228	0.7219	0.7674 / 0.7418	0.4336 $\pm$ 0.0053	0.4290 / 0.4393	0.4326
Semantic rung	0.7353 $\pm$ 0.0000	0.7353	0.7353 / 0.7353	0.2785 $\pm$ 0.0000	0.2785 / 0.2785	0.2785
AGAF	0.7793 $\pm$ 0.0116	0.7680	0.7912 / 0.7788	0.4102 $\pm$ 0.0279	0.3975 / 0.3910	0.4422
Loop, prototype consultant	0.7644 $\pm$ 0.0045	0.7642	0.7690 / 0.7601	0.4521 $\pm$ 0.0114	0.4414 / 0.4509	0.4641
Loop, trained-head consultant	0.8341 $\pm$ 0.0042	0.8332	0.8387 / 0.8304	0.5044 $\pm$ 0.0080	0.4985 / 0.5012	0.5135

Figure 4 draws the same numbers. The three fixed rungs behave differently on the two graphs. On NF-UNSW-NB15 they run AGAF, graph encoder, semantic rung. On NF-ToN-IoT they run graph encoder, AGAF, semantic rung, so AGAF falls below the rung it is meant to improve on. The semantic rung is the weakest on both, and by a much wider margin under NF-ToN-IoT’s imbalance, where the prototype scorer reaches 0.2785 against the graph encoder’s 0.4336.

The loop’s position depends on its consultant. With the prototype scorer it is above the graph encoder and the semantic rung on both datasets, below AGAF on NF-UNSW-NB15 by 0.0149, and above AGAF on NF-ToN-IoT by 0.0419. With the trained head it is the highest rung on both. The change is +0.0697 on NF-UNSW-NB15 and +0.0523 on NF-ToN-IoT. Two things changed together and the report does not separate them: the consultant, and the selected entropy percentile, which moved from 31 to 29 and from 25 to 16.

Both configurations stored their pooled out-of-fold predictions at all three seeds, so the change carries an interval computed the same way as every other difference here. It



Error bars are training-seed variance at a fixed fold partition; fold-partition variance is not measured. Open circles are the three individual seeds. The consultant change is separated on NF-UNSW-NB15 and not on NF-ToN-IoT; the two loop versions also differ in their selected top-k (31 to 29, and 25 to 16). The two panels have independent y-axes.

Figure 4: The four rungs on both datasets, with the loop drawn under each of its two consultants. Error bars are one standard deviation over three training seeds at a fixed fold partition; fold-partition variance is not measured. Open circles are the three individual seeds. The semantic rung is deterministic and carries no spread. The hatched bar is the superseded prototype-consultant loop; the arrow between the two loop bars carries an interval, given in the text. The two panels have independent y-axes and their absolute heights are not comparable.

is $+0.0702$ [$+0.0359$, $+0.1035$] on NF-UNSW-NB15, separated, and $+0.0504$ [-0.0091 , $+0.1091$] on NF-ToN-IoT, not separated. The seed-matched interval on NF-ToN-IoT does exclude zero, at $+0.0509$ [$+0.0022$, $+0.1013$]. That is the same pattern AGAF against the graph encoder shows below: a difference that survives resampling edges and stops surviving once training-seed variance is admitted. The interval measures the change as a whole. It does not attribute it to the consultant, because the entropy percentile moved with it.

Table 3 answers RQ1 and RQ2 by turning those orderings into intervals.

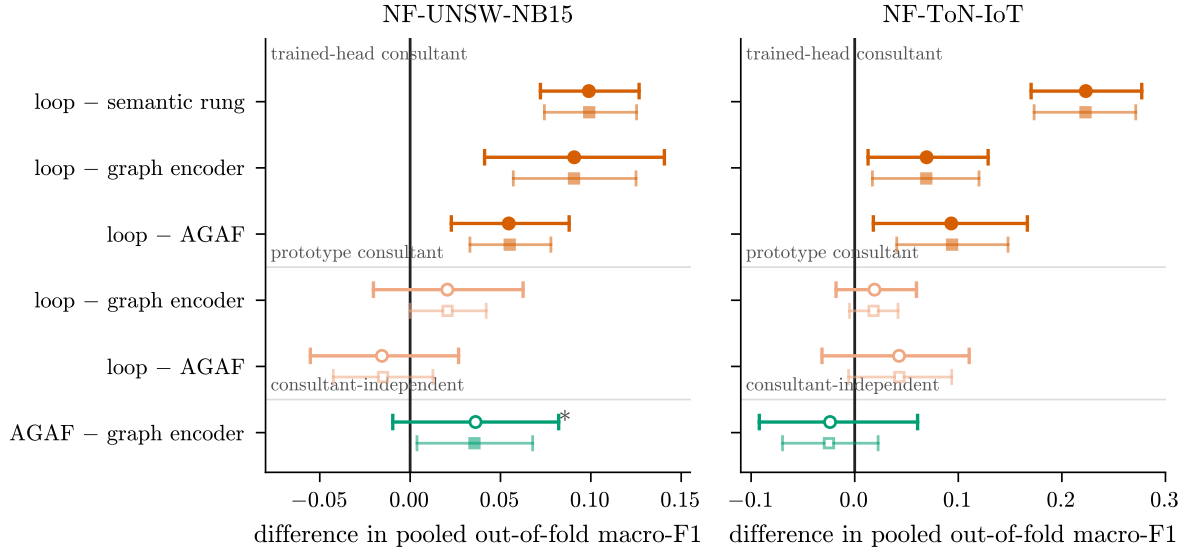
Figure 5 plots them. The table divides cleanly by consultant. Under the prototype scorer, *nothing is separated on either dataset*. Every interval crosses zero, including the two where the loop carries the higher mean. In that configuration the answer to RQ3 is that the ladder is an ordering of means and no more.

Under the trained head, every loop comparison is separated on both datasets: above AGAF, above the graph encoder and above the semantic rung, with sign stable across all three seeds. These are the first separated ladder steps this project has measured, and they hold on two datasets with the same architecture, the same folds and the same seeds.

One result is negative regardless of consultant. AGAF is not separated from the

Table 3: Rung comparisons, both bootstrap procedures, 2,000 iterations. *Sep* marks an interval that excludes zero. *Stable* reports whether the sign of the per-seed difference is the same at all three seeds. *[GAP: the prototype-consultant aggregate carries no loop versus semantic-rung comparison.]*

Dataset	Comparison	Consultant	two-level mean [95% CI]	sep	seed-matched mean [95% CI]	stable
UNSW	loop – AGAF	prototype	−0.0156 [−0.0552, +0.0268]	no	−0.0150 [−0.0425, +0.0126]	yes
	loop – AGAF	trained head	+0.0546 [+0.0228, +0.0880]	yes	+0.0551 [+0.0331, +0.0779]	yes
	loop – graph	prototype	+0.0206 [−0.0203, +0.0625]	no	+0.0207 [−0.0001, +0.0421]	yes
	loop – graph	trained head	+0.0908 [+0.0412, +0.1407]	yes	+0.0907 [+0.0571, +0.1250]	yes
	loop – semantic	trained head	+0.0989 [+0.0721, +0.1267]	yes	+0.0991 [+0.0743, +0.1253]	yes
	AGAF – graph	either	+0.0362 [−0.0096, +0.0822]	no	+0.0356 [+0.0038, +0.0678]	yes
ToN	loop – AGAF	prototype	+0.0428 [−0.0316, +0.1105]	no	+0.0431 [−0.0059, +0.0936]	yes
	loop – AGAF	trained head	+0.0932 [+0.0180, +0.1667]	yes	+0.0940 [+0.0406, +0.1480]	yes
	loop – graph	prototype	+0.0191 [−0.0180, +0.0596]	no	+0.0182 [−0.0050, +0.0418]	yes
	loop – graph	trained head	+0.0694 [+0.0129, +0.1288]	yes	+0.0691 [+0.0171, +0.1200]	yes
	loop – semantic	trained head	+0.2231 [+0.1703, +0.2771]	yes	+0.2228 [+0.1732, +0.2713]	yes
	AGAF – graph	either	−0.0238 [−0.0921, +0.0607]	no	−0.0249 [−0.0697, +0.0226]	no



* On NF-UNSW-NB15, AGAF minus graph encoder is separated under the seed-matched procedure and not once training-seed variance is admitted; the two-level result decides. There is no loop-versus-semantic-rung comparison for the prototype consultant.

● two-level (edges and training seed) ■ seed-matched (edges only) ○ interval includes zero

Figure 5: Every ladder comparison as an interval, under each consultant. A difference counts as separated only when the interval excludes zero, shown by a filled marker. The primary two-level procedure resamples edges and training seed; the seed-matched procedure resamples edges alone. Under the trained head every loop comparison clears zero on both datasets; under the prototype consultant none does. Colours follow the rung the comparison is about, with the prototype group in a lighter tint.

graph encoder: positive on NF-UNSW-NB15 and negative on NF-ToN-IoT, with an interval crossing zero either way. The seed-matched interval for NF-UNSW-NB15 does exclude zero, $+0.0356 [+0.0038, +0.0678]$. That is the clearest illustration in this report of what the two-level procedure is for: once seed variance enters, the same difference stops being separated. Under the pre-registered rule, AGAF versus the graph encoder is not separated, and the answer to RQ2 is negative on both datasets.

One caution belongs with the separated results above. They compare whole systems, and the loop differs from AGAF in two ways at once: it re-injects semantic evidence into the edge representations, and it fuses semantic logits at the output. Section 4.3 isolates the two and shows which one carries the difference. No statement of the loop’s margin over AGAF or the graph encoder in this report appears without that isolation result beside it.

4.3 Feedback ablations and the mechanism in isolation

RQ3 asks whether the feedback stage adds anything beyond static fusion, and if so, through which path. Two experiments answer it, and they answer differently.

The first is the ladder-level ablation, in which the full system runs with real advice, with the advice channel switched off, and with random advice. The feedback-off arm is called `head_only` in the code because the classifier then sees only the graph model’s own prediction head. Fusion at the output is active in all three arms, and the loop here is the canonical one, consulting the trained head. Table 4 gives every seed.

Table 4: Feedback ablation, per training seed, fusion active in all arms. The interval is a paired bootstrap of real against `head_only`.

Dataset	Seed	real	<code>head_only</code>	random	real – <code>head_only</code> [95% CI]
UNSW	42	0.8332	0.7543	0.6385	$+0.0792 [+0.0409, +0.1185]$
	1	0.8387	0.7336	0.6511	$+0.1054 [+0.0671, +0.1437]$
	2	0.8304	0.7314	0.6378	$+0.0994 [+0.0595, +0.1387]$
ToN	42	0.4985	0.3809	0.4002	$+0.1159 [+0.0596, +0.1751]$
	1	0.5012	0.4404	0.3790	$+0.0595 [+0.0074, +0.1148]$
	2	0.5135	0.4374	0.4067	$+0.0743 [+0.0223, +0.1280]$

Feedback on scores above feedback off at every seed on both datasets, with every interval excluding zero. Random advice costs at least 0.09 macro-F1 against real advice at every seed, so the arm is sensitive to what the advice says and not merely to the presence of an extra tensor. This comparison cannot attribute the gain, because the real arm differs from `head_only` in two ways at once: semantic evidence re-enters the

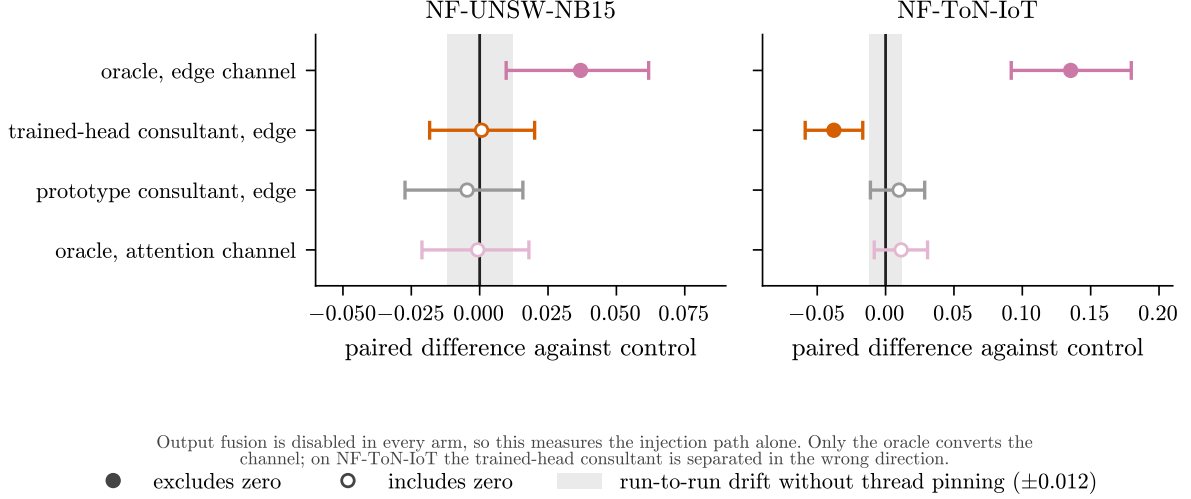


Figure 6: The injection path measured with output fusion disabled in every arm, so advice can reach the classifier only through the path under test. Filled markers mark intervals that exclude zero. The shaded band is the documented run-to-run drift without thread pinning, ± 0.012 macro-F1. Only the oracle converts the channel on both datasets; on NF-ToN-IoT the trained-head consultant is separated in the wrong direction.

edge representations, and semantic logits are fused at the output.

The second experiment separates them. Output fusion is disabled in every arm, so advice can only reach the classifier through the injection path under test. Table 5 gives the result.

Table 5: Mechanism in isolation, output fusion disabled in every arm. Means are over three seeds; intervals are paired over the fifteen (seed, fold) pairs. Captured share is the arm’s gain as a fraction of the oracle’s.

Arm	NF-UNSW-NB15		NF-ToN-IoT	
	mean	paired diff vs control [95% CI]	mean	paired diff vs control [95% CI]
control (head_only)	0.7398	–	0.4195	–
prototype, edge	0.7382	−0.0046 [−0.0273, +0.0158]	0.4325	+0.0099 [−0.0111, +0.0286]
trained head, edge	0.7441	+0.0007 [−0.0183, +0.0201]	0.3781	−0.0378 [−0.0588, −0.0167]
oracle, edge	0.7743	+0.0369 [+0.0097, +0.0618]	0.5286	+0.1354 [+0.0919, +0.1797]
oracle, attention	0.7382	−0.0007 [−0.0211, +0.0180]	0.4310	+0.0114 [−0.0083, +0.0307]
<i>Pooled headroom and captured share</i>				
oracle headroom		+0.0345		+0.1090
prototype share		−4.5%		+11.9%
trained head share		+12.5%		−38.0%

Figure 6 draws the four arms. The oracle receives the true label at ± 4 nats through the same projection every other arm uses. It is a deliberate leak, a diagnostic only and never a performance figure. It gains +0.0369 [+0.0097, +0.0618] on NF-UNSW-NB15 and +0.1354 [+0.0919, +0.1797] on NF-ToN-IoT, both separated. The channel therefore has measurable capacity on both graphs.

Neither realistic consultant uses it. The embedding-only prototype captures -4.5% of the NF-UNSW-NB15 headroom and 11.9% of the NF-ToN-IoT headroom, neither separated from zero. The trained head, which lifts the full loop by $+0.0697$ and $+0.0523$ when it replaces the prototype, captures 12.5% on NF-UNSW-NB15, again not separated, and -38.0% on NF-ToN-IoT, a separated regression. Its advantage in the full system does not survive the removal of output fusion. A cross-fitted variant of the same head, measured at seed 42 only, moves the arm from -0.0230 to -0.0654 against control on NF-UNSW-NB15 and from -0.0165 to $+0.0070$ on NF-ToN-IoT, so matching the advice’s train-time and test-time reliability does not rescue it either.

The NF-UNSW-NB15 headroom comes with a caveat. At seed 42 alone the oracle arm scores 0.7522 against a control of 0.7543 , a headroom of -0.0021 , so a captured share is not computable there. The $+0.0345$ is a three-seed mean, and single-seed shares on NF-UNSW-NB15 are not reported.

Read together, the two experiments give RQ3 a split answer. Feedback plus fusion is separated from fusion alone at every seed on both datasets. The injection mechanism alone, driven by any consultant we can actually build, is not separated from doing nothing.

4.4 Five attempts on the embedding-only consultant

Before the consultant was changed, five treatments were run against the whitened-prototype scorer. Each attacked a different candidate explanation for why its advice does not convert. All five failed, and the pattern of failure is the evidence that the constraint is the consultant rather than the mechanism around it.

Two of them were signal-quality fixes, measured at three seeds as conditions A, B and C. Condition A is the canonical configuration. Condition B trains the selector head whose entropy ranks edges, and calibrates the consultant’s temperature per fold instead of pinning it. Condition C re-selects the knobs on top of B. The two defects were genuinely removed: on NF-UNSW-NB15 the selector head’s macro-F1 moved from 0.019 – 0.036 to 0.646 – 0.684 , mean disagreement fell from 0.8975 to 0.524 – 0.548 , and the consultant’s maximum softmax probability rose from 0.102 to 0.487 , with the temperature falling from about 9.4 to 0.105 . The loop then scored 0.7644 in A, 0.7610 in B and 0.7809 in C on NF-UNSW-NB15, and 0.4521 , 0.4147 and 0.4630 on NF-ToN-IoT. Fixing both signals made the loop worse on both datasets. Condition C recovers, but its selection curves are flat, NF-UNSW-NB15’s argmax landed on the boundary of the swept range at $k = 35$, and `loop_vs_agaf` includes zero in all three conditions.

The same runs produced the clearest single diagnostic in the feedback work. The magnitude actually added to the 39-wide encoded edge vector, whose features have unit

variance, was 0.077 / 0.060 / 0.061 on NF-UNSW-NB15 and 0.023 / 0.021 / 0.023 on NF-ToN-IoT under condition A. Calibrating the temperature raised it to roughly 1.2 and 2.8 respectively, a factor of about 17 and about 125. The canonical prototype loop was, in effect, injecting almost nothing.

Table 12 in Appendix A.4 lists all five treatments and what each was worth. Appendix A gives the conditions behind the first two in full. The advice-format arm produced a counter-intuitive measurement worth stating on its own. Handing the consultant the oracle’s saturated format made the input magnitude larger and the injected magnitude smaller, from 0.0767 to 0.0183 on NF-UNSW-NB15 and from 0.0234 to 0.0202 on NF-ToN-IoT. The projection is learned, so it adapts to a larger and lower-variance input by injecting less. Whatever the oracle’s advantage is, it is not the shape of the vector it presents.

The cross-fitting arm tested the loop’s trust directly. If in-fold optimism in the advice were what the loop was exploiting, honest advice should lower the scalar the loop learns for how hard to push. Across four settings the learned trust scalar moved by at most 0.0205, from 1.5722 to 1.5664 and 1.5227 to 1.5022 on NF-UNSW-NB15 and from 1.4768 to 1.4652 and 1.4842 to 1.4899 on NF-ToN-IoT, moving the wrong way in the last of those. What did move was the injected magnitude, from 1.3430 to 1.1752 and from 1.3291 to 1.0745. The model absorbs the change in the projection rather than in the parameter that is supposed to represent trust.

Finally, the consultants differ in how well they complement the graph encoder on the edges the loop actually consults. Appendix B.3 tabulates that. On the flagged set at seed 42 the graph encoder is 0.547 accurate on NF-UNSW-NB15 and 0.343 on NF-ToN-IoT. The prototype scorer reaches 0.647 and 0.284 there, so on NF-ToN-IoT it is worse than the model it is meant to correct. The trained head reaches 0.795 and 0.648.

The disagreements are where a consultant either earns its place or does not. Among flagged edges where the consultant and the graph encoder disagree, the trained head is right 55 times against 22 on NF-UNSW-NB15 and 129 against 75 on NF-ToN-IoT. Inside the confidence-gated half those become 17 against 4 and 79 against 13. The prototype scorer is right 38 against 45 on NF-UNSW-NB15 and 45 against 178 on NF-ToN-IoT. On the edges it is asked about, and in the cases where its answer would change something, that consultant is wrong roughly four times in five.

4.5 Attention injection

The alternative injection design biases graph attention rather than the edge representation. It produces churn of exactly 0.0000 in every configuration tested, including when

handed the true label at ± 4 nats. Not one prediction changes. Table 5 confirms it a third time under the current encoding: the oracle attention arm is -0.0007 $[-0.0211, +0.0180]$ on NF-UNSW-NB15 and $+0.0114$ $[-0.0083, +0.0307]$ on NF-ToN-IoT, neither separated from zero.

The cause is structural and holds independently of the encoding. Attention modifies node embeddings. On NF-UNSW-NB15, 58.7% of edges share a source and destination pair with at least one other edge, so node-derived terms are identical for them and only the edge attributes distinguish them, and attention never touches those. On NF-ToN-IoT a different mechanism produces the same outcome: 62.8% of edges have a destination of in-degree one, where a softmax over incoming edges is 1.0 whatever bias is applied. Two datasets, two independent reasons, the same null.

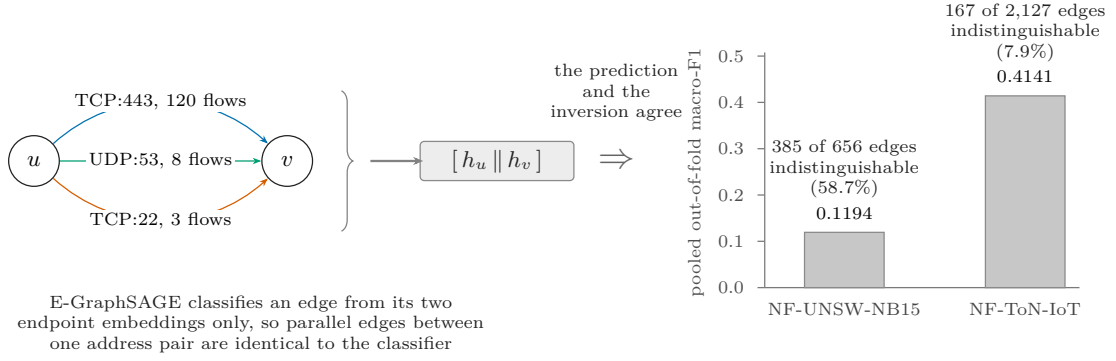
4.6 Re-trained published baselines

RQ5 compares the system against two published graph-based intrusion detection architectures, E-GraphSAGE and TE-G-SAGE, re-trained on the same aggregated rows, folds, evaluation classes and metric. These are not comparisons against the papers’ published numbers, which were obtained on per-flow graphs roughly 300 times larger. Table 6 gives the point estimates.

Table 6: Re-trained baselines, mean pooled out-of-fold macro-F1 over seeds 42, 1 and 2. *Faithful* marks a configuration that is the published architecture; **+ node features** is a labelled ablation of ours and must not be tabulated under the paper’s name.

Architecture	Configuration	Faithful	NF-UNSW-NB15	NF-ToN-IoT
E-GraphSAGE	as published	yes	0.1194 ± 0.0065	0.4141 ± 0.0046
	refit	yes	0.1368 ± 0.0076	0.4141 ± 0.0046
	+ our node features	no	0.1173 ± 0.0065	0.4358 ± 0.0323
TE-G-SAGE	as published	yes	0.3841 ± 0.0078	0.1931 ± 0.0161
	refit	yes	0.5842 ± 0.0224	0.2345 ± 0.0101
	+ our node features	no	0.7196 ± 0.0094	0.3523 ± 0.0182

The collapse of E-GraphSAGE on NF-UNSW-NB15 is a representation effect with a testable prediction attached, not evidence that the architecture is weak. Its edge classifier reads only the concatenation $[h_u || h_v]$ of the two endpoint embeddings. An edge’s own attributes never reach it, so parallel edges between one address pair cannot be told apart. That predicts a collapse proportional to how common parallel edges are. On NF-UNSW-NB15, 385 of 656 edges are indistinguishable by endpoints, that is 58.7%, and the architecture scores 0.1194. On NF-ToN-IoT only 167 of 2,127 edges are, that is 7.9%, and the same architecture scores 0.4141. The prediction and the inversion



Not a training-schedule artifact: at the authors' full 4,999 epochs on fold 0, validation macro-F1 plateaus near 0.11. Both bars are drawn in grey because neither is one of our rungs.

Figure 7: Why E-GraphSAGE collapses on one dataset and not the other. Its edge classifier reads only the concatenated endpoint embeddings, so parallel edges between one address pair are identical to it. The share of edges that are indistinguishable in this sense differs by roughly a factor of seven between the two graphs, and the scores invert with it. Both bars are grey because neither is one of our rungs.

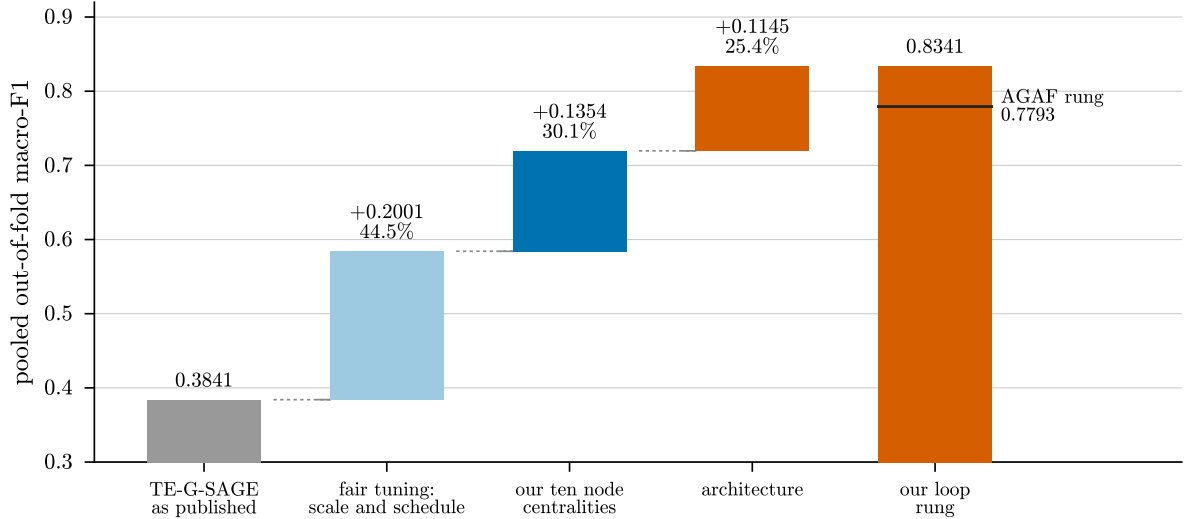
agree. Figure 7 shows the mechanism beside the confirmation. The ceiling is also not a training-schedule artifact: trained to the authors' full 4,999 epochs on fold 0, validation macro-F1 plateaus near 0.11 and peaks at 0.1384 at epoch 1,400.

Table 7 gives the intervals for the two strongest rungs.

Table 7: The loop against each re-trained baseline, under both consultants, two-level bootstrap over edges and both seed sets. Positive favours our rung. *ns* marks an interval containing zero.

Dataset	Baseline	Loop, prototype consultant	Loop, trained-head consultant
UNSW	E-GraphSAGE, as published	+0.6428 [+0.5993, +0.6868]	+0.7130 [+0.6708, +0.7540]
	E-GraphSAGE, refit	+0.6257 [+0.5807, +0.6701]	+0.6960 [+0.6526, +0.7373]
	E-GraphSAGE + features	+0.6454 [+0.6005, +0.6901]	+0.7157 [+0.6731, +0.7558]
	TE-G-SAGE, as published	+0.3798 [+0.3257, +0.4336]	+0.4500 [+0.3924, +0.5042]
	TE-G-SAGE, refit	+0.1817 [+0.1216, +0.2454]	+0.2519 [+0.1914, +0.3169]
	TE-G-SAGE + features	+0.0453 [+0.0033, +0.0847]	+0.1155 [+0.0673, +0.1627]
ToN	E-GraphSAGE, as published	+0.0378 [-0.0098, +0.0857]	<i>ns</i> +0.0890 [+0.0312, +0.1480]
	E-GraphSAGE, refit	+0.0378 [-0.0098, +0.0857]	<i>ns</i> +0.0890 [+0.0312, +0.1480]
	E-GraphSAGE + features	+0.0156 [-0.0616, +0.0908]	<i>ns</i> +0.0667 [-0.0181, +0.1505] <i>ns</i>
	TE-G-SAGE, as published	+0.2572 [+0.2010, +0.3170]	+0.3083 [+0.2345, +0.3793]
	TE-G-SAGE, refit	+0.2161 [+0.1624, +0.2688]	+0.2672 [+0.2015, +0.3348]
	TE-G-SAGE + features	+0.0990 [+0.0425, +0.1601]	+0.1502 [+0.0749, +0.2268]

With the trained-head consultant the loop is separated above every faithful re-trained baseline on both datasets. The only cells that are not separated involve **+ our node features**, our own ablation and not a published architecture, and even those are separated under the seed-matched procedure. With the prototype consultant the picture is weaker in one place that matters: on NF-ToN-IoT the loop is not separated from E-GraphSAGE in any of its three configurations. The consultant change is what moves



Point estimates only; every level is a three-seed mean and no step carries an interval or a separation claim.

Figure 8: The NF-UNSW-NB15 margin against TE-G-SAGE as published, decomposed. Roughly 45% of the gap is our choice of how to tune, 30% is our ten node centralities, and 25% is the architecture. The tick on the final increment marks where AGAF lands on the same scale. Every level is a three-seed mean; no step carries an interval and no step is a separation claim.

those three cells across the line.

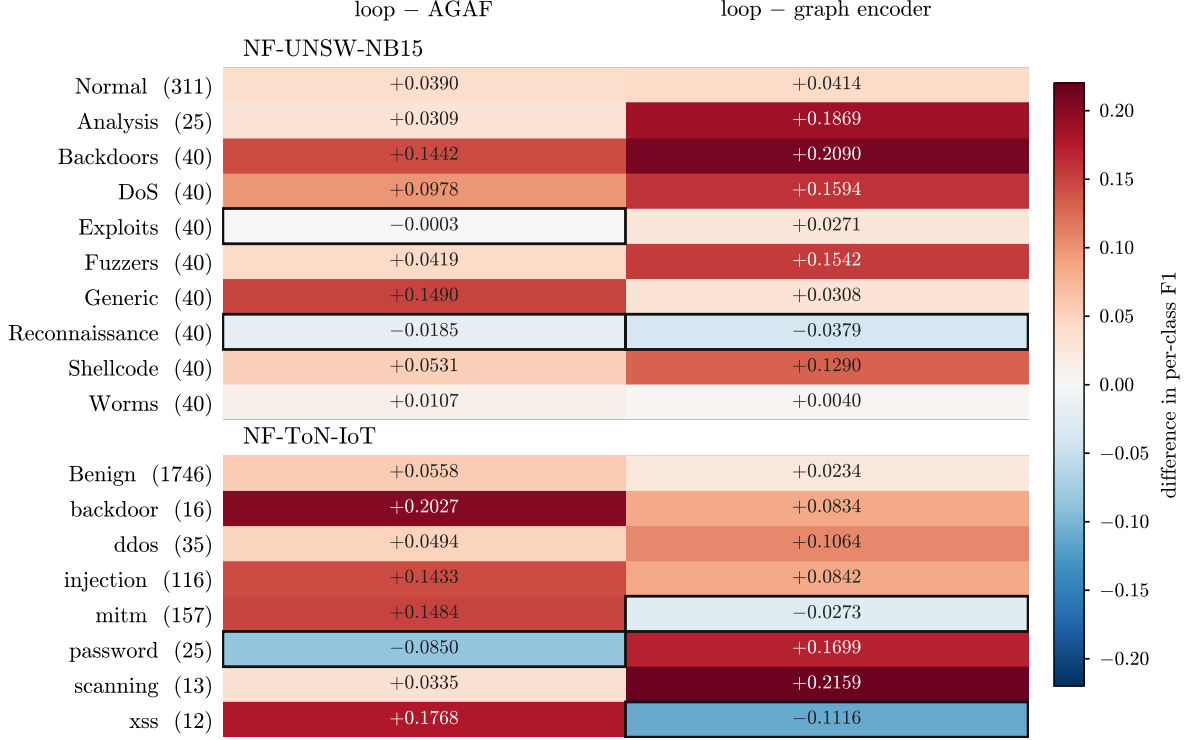
The lower rungs are weaker still. On NF-ToN-IoT neither the graph encoder nor AGAF is separated from E-GraphSAGE as published, at $+0.0203$ $[-0.0230, +0.0633]$ and -0.0061 $[-0.0797, +0.0777]$. On NF-UNSW-NB15 neither the graph encoder nor the semantic rung is separated from TE-G-SAGE with our node features, at $+0.0249$ and $+0.0163$.

Because most of the NF-UNSW-NB15 margin is not architectural, it repays decomposition. Table 8 walks from the strongest published configuration to our rungs. Every level is a three-seed mean, no step carries an interval, and no step is a separation claim.

Table 8: Decomposition of the NF-UNSW-NB15 gap against TE-G-SAGE. Point estimates only. Shares are of the total gap to the loop.

Step	macro-F1	attributable to	share
TE-G-SAGE as published	0.3841	—	—
+ fair tuning (refit)	0.5842	+0.2001, graph scale and schedule	44.5%
+ our ten centralities	0.7196	+0.1354, feature engineering	30.1%
our loop rung	0.8341	+0.1145, architecture	25.4%
(our AGAF rung)	0.7793	+0.0597 over the same base	15.1% of 0.3952

Figure 8 draws the walk. Roughly 45% of the naive gap is our choice of how to tune, roughly 30% is our node features, and roughly 25% is the architecture. Measured to



Three-seed means. Class sizes in brackets. A black outline marks a loss. No per-class difference carries an interval, and on classes of 12 to 40 edges a single edge moves F1 substantially.

Figure 9: Per-class differences between the loop and the two rungs below it, three-seed means, with class sizes in brackets. The scale is centred at zero and a black outline marks a loss. No per-class difference carries an interval, and on classes of 12 to 40 edges a single edge moves F1 substantially.

AGAF instead of the loop, the architecture share is 15.1%.

4.7 Per-class behaviour

Macro-F1 averages over classes of very different sizes. Table 13 in Appendix B.1 gives the per-class picture with the class sizes beside it. Figure 9 gives the differences the Discussion argues about.

The loop is above AGAF on eight of ten NF-UNSW-NB15 classes. The two exceptions are Reconnaissance, 0.8731 against 0.8916, and Exploits, where the two are level to three decimals. On NF-ToN-IoT the loop is above AGAF on seven of eight, losing only **password** at 0.2100 against 0.2950. Against the graph encoder the loop is above on nine of ten NF-UNSW-NB15 classes and six of eight NF-ToN-IoT classes, losing Reconnaissance on the first and **mitm** and **xss** on the second.

RQ4 turns on two rare-class observations. AGAF’s NF-ToN-IoT collapse is concentrated in the classes where the graph encoder is strong: **mitm** falls from 0.8838 to 0.7082 and **xss** from 0.3728 to 0.0844. Meanwhile AGAF is the only rung that lifts **password**

above 0.29 from the graph encoder’s 0.0401. On the smallest classes, single edges move F1 by large amounts, and none of these per-class differences carries an interval.

4.8 Knob selection

Two quantities are selected per consultant and per dataset on validation folds only, never on test: the entropy percentile that flags edges and the injection scale. For the trained-head consultant the selected values are $k = 29$ and scale 2.0 on NF-UNSW-NB15 and $k = 16$ and scale 20.0 on NF-ToN-IoT. The prototype consultant’s ladder ran at $k = 31$ and $k = 25$ at the same two scales. With the confidence gate keeping the top half of the flagged set, the effective consultation rate is 14.5% of edges on NF-UNSW-NB15 and 8.0% on NF-ToN-IoT. In edges that is 190 of 656 flagged and 95 gated on NF-UNSW-NB15, and 341 of 2,127 flagged and 170 gated on NF-ToN-IoT.

Neither quantity is tuned, and the report does not describe them that way. Averaged over the three selection seeds, the 21 candidates for k span 0.8238 to 0.8277 of validation macro-F1 on NF-UNSW-NB15 and 0.5170 to 0.5232 on NF-ToN-IoT. The six injection-scale candidates span 0.8252 to 0.8277 and 0.5196 to 0.5232. Table 14 in Appendix B.2 gives every curve. Every one of those spans is smaller than the rungs’ own seed-to-seed spread, so the argmax is not distinguishable from its neighbours and the selection is picking noise. NF-ToN-IoT’s scale selected on the upper boundary of the swept range at 20.0, and the sweep therefore cannot say whether an optimum lies outside it. The contract records that boundary condition explicitly.

5 Discussion

5.1 Answers to the research questions

RQ1. Measured on their own, the two branches are close on NF-UNSW-NB15 and far apart on NF-ToN-IoT. The graph encoder reaches 0.7437 and 0.4336, the semantic rung 0.7353 and 0.2785. At face value that suggests the semantic view is nearly worthless under heavy imbalance. The consultant experiment shows the reading is wrong: holding the embeddings, the graph, the folds and the seeds fixed, and changing only the classifier placed on those embeddings, lifts the loop by +0.0697 and +0.0523. The semantic branch’s weakness as a rung therefore traces to the whitened prototype scorer, not to the encoder underneath it. Neither modality is inherently weaker, and the read-out chosen over the semantic embeddings matters more than the choice between the two views.

RQ2. No. AGAF is not separated from the graph encoder on either dataset: +0.0362 [−0.0096, +0.0822] on NF-UNSW-NB15 and −0.0238 [−0.0921, +0.0607] on NF-ToN-IoT. The point estimates even disagree in sign. Under the seed-matched procedure the NF-UNSW-NB15 difference is separated, and stating that is more useful than hiding it, because it locates the uncertainty precisely. At a fixed pair of training seeds the fusion stage looks like an improvement; admit training-seed variance and it stops looking like one. Under the rule fixed before the runs, gated attention fusion does not deliver a separated gain over the structural branch it fuses.

RQ3. The answer splits, and the split is the finding. With the mechanism as a whole, feedback adds something: real advice is above `head_only` at every seed on both datasets, with every interval excluding zero. With the mechanism isolated, by disabling output fusion so advice can only arrive through the injection path, it adds nothing that survives a bootstrap. The embedding-only consultant is at −0.0046 and +0.0099 against control, and the trained head is at +0.0007 and −0.0378, the last of those a separated regression. The ladder-level ablation measures fusion and injection together; the isolation experiment shows which of the two carries it. The loop’s advantage over feedback-off is consistent with semantic evidence being available at the classifier at all, not with the re-entry path being what converts it.

RQ4. Three of the four factors measured bind. Topology decides whether the attention variant can work at all: 58.7% of NF-UNSW-NB15 edges share an address pair, 62.8% of NF-ToN-IoT edges have a destination of in-degree one, and in both regimes

the injected bias cannot change a prediction. Repeated endpoint pairs also set a ceiling for any architecture that classifies edges from their endpoints alone, which is why E-GraphSAGE scores 0.1194 where 385 of 656 edges are indistinguishable and 0.4141 where only 167 of 2,127 are. Class imbalance decides whether fusion survives, as discussed below. The fourth factor is consultant strength, and the mechanism turned out to depend on it entirely.

RQ5. Against published architectures re-trained on the same rows, folds and metric, the loop with its trained-head consultant is separated above every faithful configuration on both datasets. With the prototype consultant it is not separated from E-GraphSAGE on NF-ToN-IoT. The other rungs are weaker still. On NF-ToN-IoT neither the graph encoder nor AGAF is separated from E-GraphSAGE as published; on NF-UNSW-NB15 neither the graph encoder nor the semantic rung is separated from TE-G-SAGE carrying our node features. The decomposition says how much of the margin is architectural. Of the gap between TE-G-SAGE as published and our loop rung on NF-UNSW-NB15, roughly 45% is attributable to tuning the graph scale and schedule, roughly 30% to our node features, and roughly 25% to the architecture. Measured to AGAF instead the architectural share is 15.1%. A large headline margin against a published number is mostly a statement about the representation both models were given.

5.2 Imbalance is the axis the two datasets differ on

The two graphs were chosen to differ by roughly a factor of 47 in most-to-least-frequent class ratio, drawn in Figure 2, and that difference is where the ladder changes shape. AGAF is the stage that does not survive it, sitting above the graph encoder on NF-UNSW-NB15 and below it on NF-ToN-IoT. The per-class table shows the damage concentrated in classes the graph encoder already handles well: `mitm` falls from 0.8838 to 0.7082 and `xss` from 0.3728 to 0.0844. A gate that learns per-edge modality weights has to learn them from the same scarce rare-class edges it is meant to help, and on a graph where seven attack classes hold 35 edges or fewer there is little to learn from. The harm is not uniform: AGAF is the only rung that lifts `password` from 0.0401 to 0.2950. The stage trades one set of rare classes for another; it does not fail outright.

The loop does not show that trade, under either consultant. It stays above the graph encoder on both datasets and keeps its position when the imbalance regime changes. Differing class counts permit a comparison of shapes and not of levels, and on that comparison the property of collapsing under imbalance belongs to the fusion gate, not to the use of semantic evidence as such.

5.3 Where the loop’s gain comes from

The loop is separated above the three other rungs, and the isolation experiment says the separation is not produced by the part of the design the work set out to test. Four observations bear on that, and they point the same way.

First, the loop reaches the classifier by two routes and only one of them is the feedback mechanism. Output fusion places the consultant’s logits into the final classifier directly. The injection path modifies the edge representations the next graph pass consumes. Table 5 disables the first and measures the second, and with either consultant the second contributes nothing separated from zero.

Second, only advice that is always right converts. The oracle, handed the true label through the same learned projection every other arm uses, gains +0.0369 and +0.1354, both separated. Its advantage is not the format of the vector it presents: handing a realistic consultant that exact format made things worse, and drove the injected magnitude down from 0.0767 to 0.0183 on NF-UNSW-NB15. The projection is learned, so it responds to a saturated input by injecting less.

Third, the loop does not learn a trust it could calibrate. Cross-fitted advice is honestly less reliable than in-fold advice, and substituting it moved the learned trust scalar by at most 0.0205 across four settings, the wrong way in one of them. What moved instead was the injected magnitude, from 1.3430 to 1.1752 and from 1.3291 to 1.0745. The parameter that is supposed to represent how much the loop believes its consultant is not doing that job.

Fourth, five treatments failed from five different directions. Supervising the selector head, calibrating the consultant’s temperature, weighting advice by measured per-class reliability, saturating the advice format, and matching the advice’s train-time and test-time reliability were each tried and each reverted. Selection, weighting, format and reliability matching are the obvious levers, and none of them is the binding one. Changing the consultant itself did work: the sixth attempt, and the only successful one.

One diagnostic from the earlier configuration sharpens the point. Under the prototype consultant the magnitude actually injected into unit-variance features was 0.077 and 0.023. The mechanism was, in effect, switched off, and the ladder measured at the time was measuring fusion while describing feedback. That is why this report separates the two paths everywhere, and why the isolation experiment, not the ladder, answers RQ3.

5.4 What the configuration buys, and what it does not

Two statements have to be held together. The trained-head configuration produces the first ladder steps this project has measured that survive training-seed variance. It produces them on two datasets, under one architecture, with the same folds and the same seeds. The loop is separated above AGAF, above the graph encoder and above the semantic rung, and above every faithful re-trained baseline. Under the prototype consultant none of those comparisons is separated on either dataset, so the consultant is what turned an ordering of means into a result.

What the configuration does not buy is a separated gain from the feedback mechanism itself. With output fusion disabled, so that advice can reach the classifier only by re-entering the edge representations, neither consultant is separated from doing nothing, and on NF-ToN-IoT the stronger one is separated in the wrong direction. The system works. The part of it this work set out to test does not carry the result.

The contribution, stated narrowly, is instantiation plus measurement. DAS reports consistent gains from a closed feedback loop between a frozen graph model and a language model, at node level on text-rich graphs. This work instantiates the same shape of mechanism at edge level on network-flow graphs, with a deliberately constrained consultant that sees one edge’s text and nothing else, and finds that the refinement result does not transfer. It then isolates why, and that isolation is the part that generalises: the channel has demonstrable capacity, an oracle converts it on both datasets, and no consultant we were able to build does. LOGIN, GLANCE and RoGRAD each establish a component of the design (uncertainty-driven consultation, learned routing and iterative refinement respectively), and none of them is claimed here as novel. What is new is the measurement, in a setting where the answer turns out to be negative.

5.5 Limitations, read as findings

Most of the seven limitations below are results in their own right, and all of them affect how the numbers should be read.

1. **Edge identity depends on the label.** Flows are aggregated by source IP, destination IP and attack class, so ground truth participates in defining what an edge is. Every number in this report is a controlled comparison between representations and mechanisms, and none is a deployment-valid detection estimate. This is the limitation that most restricts what the work can claim.
2. **Only training-seed variance is measured.** The fold partition is fixed at the seed-42 split, so every reported standard deviation and every interval is a lower

bound on total uncertainty. The two comparisons this report calls not separated are already close to zero, and measuring partition variance could only widen them.

3. **The loop trains on advice more reliable than the advice it is scored on.** Each fold’s head has fitted that fold’s training labels, so on training edges its logits are in-fold. Cross-fitting removes the asymmetry and makes the loop worse on both datasets, so the effect is not a simple optimism bias that honest advice would fix.
4. **Neither knob is tuned.** Both selection curves lie inside their own noise band, and NF-ToN-IoT’s injection scale selected on the upper boundary of the swept range, so the sweep cannot say whether an optimum lies outside it. The word used throughout is selected.
5. **One consultant file sits inside the loop at all three seeds.** The trained head was built once, at seed 42, and the loop consults that same file at every training seed. Its advice is identical across the three runs. The loop’s reported standard deviation, 0.0042 and 0.0080, excludes consultant variance entirely and is narrower than a fully re-seeded pipeline would give.
6. **The isolation experiment’s pairs are not independent.** Its intervals are computed over fifteen (seed, fold) pairs, and the five folds within a seed share a training run. The effective sample size is smaller than fifteen, so those intervals are optimistic in the same direction for every arm.
7. **E-GraphSAGE is within noise of our structural rungs on NF-ToN-IoT.** Neither the graph encoder nor AGAF is separated from it there. Only the two rungs that use a trained semantic consultant clear it, and only at the second consultant. The honest summary of RQ5 is that our structural pipeline matches a published architecture on that graph rather than improving on it.

5.6 What follows from this

Three directions follow directly from this, along with one non-direction worth recording.

Trust is the parameter to work on next. The loop has a scalar meant to represent how much to believe the consultant, and it does not respond to a large, deliberate change in the consultant’s reliability. Constraining that scalar, supervising it against measured per-edge reliability, or replacing it with a per-edge quantity are all reachable experiments. The oracle result says there is something to win if the consultant can be made trustworthy where it is confident.

Deployment-valid graph construction is the prerequisite for any operational claim. It requires an aggregation key that does not read the label, which changes the edge set, the class distribution and the difficulty of the task all at once. That is a new set of experiments rather than a re-run of these.

A stronger consultant is a reasonable third direction, with one caveat this work supplies: standalone accuracy is not the property that matters. Through the injection path on NF-ToN-IoT the stronger consultant is the worse one, at -0.0378 against control where the weaker one sits at $+0.0099$. Calibration on the flagged subset, not accuracy over the whole graph, is what a replacement consultant would have to improve.

The non-direction is advice that depends on the graph model’s current belief. The consultant here sees one edge’s text and nothing else, which is what makes the coupling bidirectional in routing and not in content. Conditioning the advice on the graph model’s top candidates would very likely change the result, and it was excluded by design so that the semantic branch’s contribution could be attributed to it alone. A future system that removes that constraint is testing a different question, and it should say so.

6 Conclusion

This work built one reproducible edge-level intrusion detection pipeline and ran it unchanged on two NetFlow datasets that differ by roughly a factor of 47 in class imbalance. Each aggregated flow is encoded twice, once structurally by a graph attention encoder and once as a label-free sentence read by a frozen cybersecurity language encoder. Four stages are compared under one protocol: each branch alone, gated attention fusion, and a feedback loop that consults the semantic branch about high-entropy edges and returns the judgement into the edge representations. The loop is reported under two consultants, an embedding-only prototype scorer and a classification head trained on the same semantic embeddings, because the change between them turned out to be the largest effect measured.

Everything was measured at three training seeds with the fold partition fixed, scored by pooled five-fold out-of-fold macro-F1, and compared with a bootstrap that resamples edges and training seed. A difference counts as separated only when its interval excludes zero. Two sentences carry the result.

With the trained-head consultant the loop is separated above the fusion stage, above the graph encoder, above the semantic rung and above every faithful re-trained baseline, on both datasets. With the prototype consultant, none of those comparisons is separated on either dataset. Changing the consultant, and nothing else about the architecture, moved the loop by +0.0697 and +0.0523.

The second result explains the first. An oracle consultant, handed the true label through the same projection, converts the feedback channel into a separated gain on both datasets, so the channel has real capacity. No consultant we could build exploits it. Five treatments were tried and all five reverted, the learned trust scalar barely moves when the consultant’s reliability is changed deliberately, and with output fusion disabled neither realistic consultant is separated from doing nothing. The binding constraint is the consultant, not the mechanism around it.

Two findings stand on their own. Gated attention fusion is not separated from the graph encoder on either dataset and falls below it under heavy imbalance. The alternative injection design, which biases graph attention instead of the edge representation, changes no prediction at all, for two independent structural reasons on the two graphs.

What none of this establishes is deployment performance. Edges are aggregated with the attack label in the grouping key, so these numbers compare representations and mechanisms under a controlled protocol. Removing that dependency is the prerequisite for any operational claim, and it changes the task rather than the code. Beyond it, what would matter is not a larger consultant but a better calibrated one, with a trust parameter that responds when reliability changes. The oracle says the headroom is

there. Nothing measured here has reached it.

Acknowledgements

I thank my supervisor, Dr. Hussein Hazimeh, for guidance throughout this work, and Dr. Fouad Al Tfaily, who supervised the work day to day at CESI LINEACT on his behalf. I thank CESI LINEACT and the Lebanese University for hosting it. The two datasets are used in the NetFlow-standardised forms released by their authors, and the two baseline architectures were re-implemented from their published descriptions and released code.

Several of this report’s findings exist because a claim was checked and did not survive. That habit cost time and improved the work, and the record of what was withdrawn is kept alongside the record of what stood.

Statement on the use of AI assistance

AI assistance was used for drafting and editing prose in this report, for writing and refactoring pipeline and analysis code, and for building the scripts that audit the manuscript’s numbers against the committed result artifacts. Every experiment was designed and run by the author. Every number reported here comes from a committed artifact produced by those runs, and no figure, table or quantitative claim was generated from a model’s recollection. The interpretation of the results, the decisions about what to measure, what to retract and what to report as negative, and the final text are the author’s own.

References

- [1] John Arévalo, Thamar Solorio, Manuel Montes y Gómez, and Fabio A. González. Gated multimodal units for information fusion, 2017. arXiv:1702.01992. Verified via Scite 2026-09-07.
- [2] Markus Bayer, Philipp Kuehn, Ramin Shanehsaz, and Christian Reuter. Cy-SecBERT: A domain-adapted language model for the cybersecurity domain, 2022. arXiv:2212.02974.
- [3] Azadeh Beiranvand and Seyed Mehdi Vahidipour. Integrating structural and semantic signals in text-attributed graphs with BiGTex, 2025. arXiv:2504.12474. Verified via Scite and arXiv 2026-09-07.
- [4] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022. doi: 10.48550/arXiv.2105.14491. Verified via Scite 2026-09-05; no editorial notices.
- [5] Yasir Ali Farrukh, Syed Wali, Irfan Khan, and Nathaniel D. Bastian. XG-NID: Dual-modality network intrusion detection using a heterogeneous graph neural network and large language model, 2024. arXiv:2408.16021.
- [6] Lingbing Guo, Zhuo Chen, Jiaoyan Chen, Qiang Zhang, and Huajun Chen. DET: A dual-encoding transformer for relational graph embedding. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 4685–4696, 2024. doi: 10.63317/29cd7cmbyvh5. Verified via Scite, Crossref and the ACL Anthology 2026-09-07.
- [7] Hong Huang, Xingxing Zhang, Ye Lu, Ze Li, and Shaohua Zhou. BSTFNet: An encrypted malicious traffic classification method integrating global semantic and spatiotemporal features. *Computers, Materials & Continua*, 78(3):3929–3951, 2024. doi: 10.32604/cmc.2024.047918. Author list per Crossref 2026-09-07 (Scite lists the first author as Hongwei Huang; check against the PDF before submission).
- [8] Jaewan Lee, Changyoung Park, Hongjun Yang, Sungbin Lim, Woohyung Lim, and Sehui Han. CAST: Cross attention based multimodal fusion of structure and text for materials property prediction, 2025. arXiv:2502.06836. Verified via Scite and arXiv 2026-09-07.
- [9] Haolin Li, Shuyang Jiang, Lifeng Zhang, Siyuan Du, Guangnan Ye, and Hongfeng Chai. RAGFormer: Learning semantic attributes and topological structure for fraud detection, 2024. arXiv:2402.17472. Verified via Scite and arXiv 2026-09-07.

- [10] Yuxiao Lin, Yuxian Meng, Xiaofei Sun, Qinghong Han, Kun Kuang, Jiwei Li, and Fei Wu. BertGCN: Transductive text classification by combining GNN and BERT. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 1456–1462, 2021. doi: 10.18653/v1/2021.findings-acl.126. Verified via Scite 2026-09-07.
- [11] Wai Weng Lo, Siamak Layeghy, Mohanad Sarhan, Marcus Gallagher, and Marius Portmann. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. In *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, pages 1–9, 2022. doi: 10.1109/NOMS54207.2022.9789878. Verified via Scite 2026-09-05; no editorial notices. Code: github.com/waimorris/E-GraphSAGE.
- [12] Donald Loveland, Yao-An Yang, and Danai Koutra. Glance for context: Learning when to leverage LLMs for node-aware GNN-LLM fusion, 2025. arXiv:2510.10849. Verified via Scite and arXiv 2026-09-07.
- [13] Riko Luša, Damir Pintar, and Mihaela Vranić. TE-G-SAGE: Explainable edge-aware graph neural networks for network intrusion detection. *Modelling*, 6(4):165, 2025. doi: 10.3390/modelling6040165. Verified via Scite and Crossref 2026-09-07; no editorial notices. Code: github.com/Ricco555/TE-G-SAGE-XAI.
- [14] Hejamadi Rama Moorthy, N. J. Avinash, N. S. Krishnaraj Rao, K. R. Raghunandan, Radhakrishna Dodmane, Jeremy Joseph Blum, and Lubna A. Gabralla. Dual stream graph augmented transformer model integrating BERT and GNNs for context aware fake news detection. *Scientific Reports*, 15:25436, 2025. doi: 10.1038/s41598-025-05586-w.
- [15] Nour Moustafa. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustainable Cities and Society*, 72:102994, 2021. doi: 10.1016/j.scs.2021.102994. Verified via Scite 2026-09-05; no editorial notices.
- [16] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6, 2015. doi: 10.1109/MilCIS.2015.7348942.
- [17] Yiran Qiao, Xiang Ao, Yang Liu, Jiarong Xu, Xiaoqian Sun, and Qing He. LOGIN: A large language model consulted graph neural network training framework, 2024. arXiv:2405.13902. Verified via Scite and arXiv 2026-09-07.

- [18] Mohanad Sarhan, Siamak Layeghy, Nour Moustafa, and Marius Portmann. Net-flow datasets for machine learning-based network intrusion detection systems. In *Big Data Technologies and Applications*, pages 117–135. Springer International Publishing, 2021. doi: 10.1007/978-3-030-72802-1_9.
- [19] Karen Scarfone and Peter Mell. Guide to intrusion detection and prevention systems (IDPS). Technical Report NIST Special Publication 800-94, National Institute of Standards and Technology, 2007.
- [20] Aarush Sinha. GMLM: Bridging graph neural networks and language models for heterophilic node classification, 2025. arXiv:2503.05763. Verified via Scite 2026-09-07.
- [21] Safal Thapaliya, Zehong Wang, Jiazheng Li, Ziming Li, Yanfang Ye, and Chuxu Zhang. Semantic refinement with LLMs for graph representations, 2025. arXiv:2512.21106v1. The v1 text names the framework DAS; the April-2026 revision renames it GES. Verified via Scite and arXiv 2026-09-07.
- [22] Zhaoyan Wang, Zheng Gao, Arogya Kharel, and In-Young Ko. Are LLMs better GNN helpers? rethinking robust graph learning under deficiencies with iterative refinement, 2025. arXiv:2510.01910; system named RoGRAD. Verified via Scite and arXiv 2026-09-07.
- [23] Lan Xiong, Liang Wan, and Jingxia Ren. Two-branch intrusion detection method based on fusion of deep semantic and statistical features. *Computers, Materials & Continua*, 87(3):18, 2026. doi: 10.32604/cmc.2026.076986. Article 18. Verified via Scite and publisher page 2026-09-07.
- [24] Haotong Yang, Xiyuan Wang, Qian Tao, Shuxian Hu, Zhouchen Lin, and Muhan Zhang. GL-Fusion: Rethinking the combination of graph neural network and large language model, 2024. arXiv:2412.06849. Verified via Scite and arXiv 2026-09-07.
- [25] Zhonghao Zhan, Huichi Zhou, and Hamed Haddadi. Poster: Enhancing GNN robustness for network intrusion detection via agent-based analysis, 2025. arXiv:2506.20806; poster at IEEE EuroS&P 2025. Verified via Scite and arXiv 2026-09-07.
- [26] Jingkang Zhang, Xinjian Fan, and Zhixin Zhao. A hybrid intrusion detection model based on dynamic spatial-temporal graph neural network in in-vehicle networks. *Scientific Reports*, 15(1):34736, 2025. doi: 10.1038/s41598-025-18401-3. Model named GCN-2-Former. Verified via Scite and Crossref 2026-09-07; no editorial notices.

A Negative mechanism results

This appendix holds the measurements behind Section 4.4. They are reported in full because the case for changing the loop’s consultant rests on them, and because each one closes a line of investigation that would otherwise look open.

A.1 Conditions A, B and C

Three configurations of the loop were run at three seeds on both datasets, with the fold partition fixed. Condition A is the canonical prototype-consultant loop. Condition B trains the selector head whose entropy ranks edges and calibrates the consultant’s temperature per fold instead of pinning it. Condition C re-selects the two knobs on top of B. The graph encoder and AGAF rungs are unaffected by any of the three and are shown for reference in Table 9.

Table 9: Pooled out-of-fold macro-F1, mean over three seeds, by condition.

Dataset	Rung	A (canonical)	B (signals fixed)	C (B, re-selected)
UNSW	loop	0.7644	0.7610	0.7809
	AGAF	0.7793	0.7793	0.7793
	graph encoder	0.7437	0.7437	0.7437
ToN	loop	0.4521	0.4147	0.4630
	AGAF	0.4102	0.4102	0.4102
	graph encoder	0.4336	0.4336	0.4336

The decision rule was fixed before the runs. It returns the same verdict in all three conditions: the loop is not separated from AGAF on either dataset. On NF-UNSW-NB15 the two-level interval for loop minus AGAF is -0.0156 $[-0.0552, +0.0268]$ in A, -0.0192 $[-0.0594, +0.0197]$ in B and $+0.0008$ $[-0.0464, +0.0447]$ in C. On NF-ToN-IoT it is $+0.0428$ $[-0.0316, +0.1105]$, $+0.0060$ $[-0.0667, +0.0739]$ and $+0.0527$ $[-0.0118, +0.1194]$. Condition C carries the highest point estimate on both datasets. That is not a separation, and its selection curves are flat, with NF-UNSW-NB15’s argmax landing on the boundary of the swept range at $k = 35$.

Both defects were genuinely removed in B. That is what makes the negative result informative rather than a failed implementation. On NF-UNSW-NB15 the selector head’s macro-F1 moved from the range 0.019 to 0.036 up to the range 0.646 to 0.684, mean disagreement fell from 0.8975 to between 0.524 and 0.548, the consultant’s maximum softmax probability rose from 0.102 to 0.487, and the temperature fell from about 9.4 to 0.105. On NF-ToN-IoT the selector head moved from 0.029 to 0.065 up to 0.293 to 0.333, disagreement fell from 0.8988 to between 0.580 and 0.638, maximum probability rose

from 0.101 to 0.500, and temperature fell from about 9.5 to 0.030. A better-calibrated consultant and a genuinely uncertainty-ranked selector made the loop worse on both datasets.

A.2 What the canonical loop was actually injecting

Table 10 gives the mean absolute magnitude added to the 39-wide encoded edge vector on flagged edges, per seed. The encoded features have unit variance, so these numbers are directly interpretable as a fraction of a standard deviation.

Table 10: Injected bias magnitude on flagged edges, per seed, by condition.

Condition	NF-UNSW-NB15	NF-ToN-IoT
A (canonical)	0.077 / 0.060 / 0.061	0.023 / 0.021 / 0.023
B (signals fixed)	1.228 / 1.171 / 1.218	2.796 / 2.854 / 2.762
C (B, re-selected)	1.103 / 1.100 / 1.158	2.845 / 2.893 / 2.992

Under the canonical prototype configuration the mechanism was injecting between 0.02 and 0.08 of a standard deviation. Calibrating the temperature raised that by a factor of roughly 17 on NF-UNSW-NB15 and roughly 125 on NF-ToN-IoT. Two things follow. The prototype loop’s advantage over feedback-off was late fusion rather than feedback. The injection scale is also close to redundant with the learned bias strength, since condition C moved the scale from 2.0 to 5.0 and from 20.0 to 10.0 while the end-to-end magnitude barely moved.

A.3 Per-class reliability of the embedding-only consultant

The reliability-weighting treatment computes, for each class, the precision of the prototype consultant’s argmax on the fold’s training edges. The weights are a more useful result than the macro-F1 deltas they produced, because they quantify the consultant’s weakness per class from training labels alone. Table 11 gives them.

Table 11: Mean per-class precision of the prototype consultant on training edges, over five folds. Class indices follow the label mapping.

Dataset	0	1	2	3	4	5	6	7	8	9
NF-UNSW-NB15	1.000	0.852	0.976	0.893	0.961	0.662	0.895	0.984	1.000	0.960
NF-ToN-IoT	1.000	0.071	0.179	0.021	0.570	0.740	0.226	0.020	0.095	0.089

On NF-UNSW-NB15 six of ten classes sit at or above 0.9 and the minimum is 0.662, so reweighting has little to reorder. On NF-ToN-IoT six of ten sit at or below 0.226.

That is the same weakness the isolation experiment measures as the binding constraint, now quantified per class.

A.4 The five treatments

Table 12: Five treatments applied to the embedding-only consultant. None was kept. Rows marked seed 42 carry no interval across seeds and are single-seed orderings only.

Treatment	What it changed	Result	Kept
Selector-head supervision	the ranking head is trained as a classifier	3 seeds: loop worse on both datasets	no
Consultant temperature	per-fold calibration instead of a pinned T	3 seeds: loop worse, jointly with the above	no
Reliability weighting	gate and magnitude scaled by per-class train precision	seed 42: +0.0057 / +0.0094, below the 0.02 rule	no
Advice format	one-hot and softmax at the oracle’s ± 4 nats	seed 42: every arm below baseline on both	no
Cross-fitted advice	train rows replaced by inner out-of-fold logits	seed 42: −0.0246 / −0.0338 against in-fold	no

B Supporting tables

B.1 Per-class F1

B.2 Knob selection curves

B.3 Consultant complementarity on the flagged set

The loop consults the edges the graph encoder is least certain about, so what matters is not a consultant’s accuracy over the whole graph but its accuracy where it is actually asked. Table 15 gives that, at seed 42.

The NF-ToN-IoT prototype row is the clearest statement of the problem. On the edges the loop selects, that consultant is less accurate than the model it is supposed to correct.

Table 16 gives what happens where the two disagree, the only place a consultant can change an answer.

The gate is doing its job for the trained head and not for the prototype. Restricting to the consultant’s own most confident half lifts the trained head from 55 against 22 to 17 against 4 on NF-UNSW-NB15 and from 129 against 75 to 79 against 13 on NF-ToN-IoT. It does not rescue the prototype on NF-ToN-IoT, where confident disagreements

Table 13: Per-class F1, mean over three seeds, with edge counts. The loop is the canonical one, consulting the trained head. NF-ToN-IoT’s `dos` and `ransomware` are excluded from the metric and omitted here.

Class	edges	graph	AGAF	loop
<i>NF-UNSW-NB15</i>				
Normal	311	0.9129	0.9152	0.9543
Analysis	25	0.4327	0.5886	0.6196
Backdoors	40	0.5929	0.6578	0.8019
DoS	40	0.5831	0.6446	0.7424
Exploits	40	0.9309	0.9583	0.9580
Fuzzers	40	0.6207	0.7330	0.7749
Generic	40	0.6591	0.5409	0.6899
Reconnaissance	40	0.9110	0.8916	0.8731
Shellcode	40	0.8669	0.9428	0.9959
Worms	40	0.9272	0.9205	0.9311
<i>NF-ToN-IoT</i>				
Benign	1746	0.9599	0.9275	0.9832
backdoor	16	0.3633	0.2439	0.4467
ddos	35	0.1848	0.2419	0.2913
injection	116	0.5436	0.4845	0.6277
mitm	157	0.8838	0.7082	0.8566
password	25	0.0401	0.2950	0.2100
scanning	13	0.0958	0.2781	0.3116
xss	12	0.3728	0.0844	0.2612

Table 14: Knob selection curves for the trained-head consultant, mean validation macro-F1 over selection seeds 42, 1 and 2. Every span is smaller than the rungs’ own seed-to-seed spread, so the argmax is not distinguishable from its neighbours.

Dataset	Knob	candidates	range	validation macro-F1	span	selected	on boundary
NF-UNSW-NB15	k (entropy percentile)	21	15–35	0.8238–0.8277	0.0038	29	no
NF-UNSW-NB15	injection scale	6	0.5–20	0.8252–0.8277	0.0025	2	no
NF-ToN-IoT	k (entropy percentile)	21	15–35	0.5170–0.5232	0.0062	16	no
NF-ToN-IoT	injection scale	6	0.5–20	0.5196–0.5232	0.0037	20	yes

Table 15: Accuracy on the entropy-flagged edge set at seed 42, for the graph encoder, each consultant, and the resulting loop.

Dataset	Consultant	graph encoder	consultant	resulting loop
NF-UNSW-NB15	prototype	0.547	0.647	0.658
	trained head	0.547	0.795	0.779
NF-ToN-IoT	prototype	0.343	0.284	0.361
	trained head	0.343	0.648	0.628

Table 16: Among entropy-flagged edges where the consultant and the graph encoder disagree, how often the consultant is right. Seed 42, at the canonical entropy percentile for each dataset.

Dataset	Consultant	flagged set		confidence-gated half	
		right	wrong	right	wrong
NF-UNSW-NB15	prototype	38	45	13	5
	trained head	55	22	17	4
NF-ToN-IoT	prototype	45	178	28	63
	trained head	129	75	79	13

are wrong more than twice as often as they are right. A gate can only concentrate a consultant’s competence; it cannot manufacture it.

Both tables are recomputed from the committed pooled out-of-fold tensors, by a script that writes its own artifact:

```
scripts/close_complementarity_split.py
results/consultant_complementarity.json
```

C Reproducibility

Environment. Python 3.14.3 in a virtual environment built from `requirements.txt`. All runs export `OMP_NUM_THREADS=1`. Without that pin, pooled macro-F1 drifts by roughly 0.012 at a fixed seed, larger than several of the differences this report measures. One earlier finding in the project was retracted because of it.

Seeds and folds. Three training seeds, 42, 1 and 2. The stratified five-fold partition is generated once at seed 42 and held fixed across all three, so the pooled out-of-fold edge set is identical between seeds and the seed-matched bootstrap is a paired comparison. Fold-partition variance is therefore not measured anywhere in this report.

Pipeline. Graph construction runs from `src.pipeline.step1.run_step1`, writing `pyg_data.pt` and `aggregated_edges.csv` under `data/<dataset>/processed/step1/`. The downstream stages run through one entry point:

```
python -m src.pipeline.common.run_pipeline -dataset <key> \
-only fusion oof prototypes top_k_sweep feedback ladder
```

with `<key>` either `unsw_nb15` or `ton_iot`. The loop consults the trained head when `train_feedback` is given `-use-llm-head`. Without that flag it reproduces the superseded prototype-consultant ladder, at the knobs kept under `prototype_superseded` in each dataset’s selected-feedback config. Out-of-fold structural embeddings must be regenerated with `step3.build_oof_gnn_embeddings` before the fusion stage whenever the graph encoder changes. The fusion stage does not do it, and will otherwise train on stale embeddings.

Result contracts. Every number in this report traces to one of the files in Table 17.

Table 17: Committed artifacts behind the report’s numbers.

Artifact	What it carries
<code>results/unsw_nb15_current.json</code> (schema 5)	UNSW ladder, ablations, knobs
<code>results/ton_iot_current.json</code> (schema 7)	ToN ladder, ablations, knobs
<code>results/cross_dataset_comparison.json</code> (schema 6)	both datasets side by side
<code>results/multiseed_ladder_v2_head.json</code>	three-seed ladder, trained-head consultant
<code>results/multiseed_ladder_v2_legacy.json</code>	three-seed ladder, prototype
<code>results/multiseed_head_per_class.json</code>	per-class F1, three-seed means
<code>results/*_sota_baselines.json</code>	re-trained baselines and intervals
<code>results/*_oracle_ceiling_v2_trained_head.json</code>	the isolation test
<code>results/knob_selection_head/</code>	the selection curves
<code>results/consultant_change_interval.json</code>	the interval on the consultant change
<code>results/consultant_complementarity.json</code>	the flagged-set and disagreement figures
<code>docs/RESULTS_ARCHIVE.md</code>	the full record, including retractions

Checks. The repository’s test suite pins the contracts against each other and against the on-disk graphs, including a test asserting that both datasets declare the same architecture. It runs with `OMP_NUM_THREADS=1 python -m pytest tests/ -q` and reports 167 passing tests with 359 passing subtests. Two further scripts guard the manuscript itself: `scripts/check_report_numbers.py` extracts every numeric token from the section sources and checks it against the contracts, the archive, the dataset statistics and an explicit allowlist. `scripts/prose_stats.py` measures the writing against the standard used in the final editing pass. Both run from `docs/research_report/build.sh`.

What is not reproducible from this repository. The raw NF-ToN-IoT and NF-UNSW-NB15 CSV sources are not committed, being 532 MB and comparable in size. The graphs built from them are, so every result here can be re-derived without re-downloading the sources.